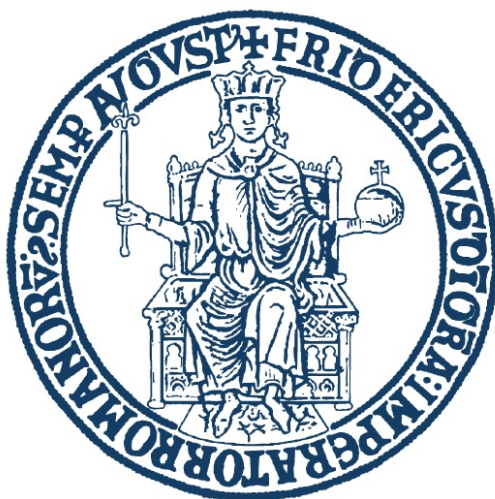




UNIVERSITÀ DEGLI STUDI DI NAPOLI FEDERICO II

UNIVERSITÀ DEGLI STUDI DI NAPOLI FEDERICO II
CORSO DI LAUREA TRIENNALE IN INFORMATICA

BugBoard26

Documentazione

Ingegneria del Software

Mario Majorano
Luca Sanselmo

N86005035
N86005147

Anno Accademico 2025/2026

Indice

Introduzione	4
1 Glossario	4
1.1 Definizioni	4
1.2 Acronimi	5
1.3 Elementi Informativi	5
2 Descrizione generale	6
2.1 Prospettiva del prodotto	6
2.2 Fondamenti dei requisiti	6
2.2.1 Funzionalità del prodotto	6
2.2.2 Stakeholder	6
2.2.3 Vincoli	7
3 Specifica dei Requisiti Software	8
3.1 Trasformazione dei bisogni in requisiti	8
3.2 Attori del sistema	9
3.3 Requisiti Specifici	10
3.3.1 Requisiti Funzionali	10
3.3.2 Requisiti Non Funzionali	14
3.4 Modellazione dei casi d'uso	18
3.4.1 Descrizione casi d'uso principali	18
3.5 Modellazione di casi d'uso	25
3.5.1 Mock-ups	25
3.5.2 Formalismi di Cockburn	27
3.6 Tracciamento dei requisiti	30
3.6.1 Tracciamento fonte-requisito	30
3.6.2 Tracciamento requisiti-casi d'uso	30
4 Design del Sistema	31
4.1 Descrizione Architettuale	31
4.2 Panoramica dell'architettura	31
4.3 Criteri di design adottati	32
4.3.1 Criteri funzionali	32
4.3.2 Criteri di qualità	32
4.3.3 Vincoli architeturali	33
4.4 Pattern architettuale adottato	33
4.4.1 Descrizione del pattern	33
4.4.2 Motivazione della scelta	33
4.5 Scelte Tecnologiche	34
4.6 Stack tecnologico	34
4.6.1 Panoramica	34
4.7 Backend	34
4.7.1 Linguaggio e framework	34
4.7.2 Motivazione	34
4.8 Frontend	35
4.8.1 Framework	35

4.8.2	Motivazione	35
4.9	Persistenza dei dati	35
4.9.1	Sistema di gestione dei dati	35
4.9.2	Motivazione	35
4.10	Comunicazione e protocolli	36
4.10.1	Protocollo di comunicazione	36
4.10.2	Formato dei dati	36
4.10.3	Motivazione	36
4.11	Sicurezza	36
4.11.1	Autenticazione e autorizzazione	36
4.11.2	Protezione dei dati	36
4.12	Interfacce API	37
4.13	Gestione degli Aspetti Non Funzionali	38
4.13.1	Performance	38
4.13.2	Scalabilità	38
4.13.3	Manutenibilità	38
5	Design del Software	39
5.1	Schema di Persistenza dei Dati	39
5.1.1	Introduzione	39
5.1.2	Progettazione Concettuale	39
5.1.3	Diagramma Concettuale	41
5.1.4	Ristrutturazione	42
5.1.5	Schema Logico-Relazionale	44
5.1.6	Script DDL	45
5.2	Diagramma delle Classi di Design	45
5.2.1	Panoramica dei package	45
5.3	Scelte di Software Design	48
5.3.1	Pattern architetturale	48
5.3.2	Design Pattern utilizzati	48
5.3.3	Repository Pattern	48
5.3.4	Dependency Injection	48
5.3.5	DTO Pattern	48
5.3.6	Singleton	48
5.3.7	Esempi di Principi SOLID	49
5.3.8	Single Responsibility Principle (SRP)	49
5.3.9	Open/Closed Principle (OCP)	49
5.3.10	Liskov Substitution Principle (LSP)	49
5.3.11	Gestione degli errori e delle eccezioni	50
5.3.12	Strategie di sicurezza applicativa	50
5.4	Versioning e Repository	50
5.4.1	Strategia di branching	50
5.4.2	Frequenza dei commit nel tempo	50
5.5	Qualità del Codice	51
5.5.1	Strumento di analisi: SonarQube	51
5.6	Build Automation	51
5.6.1	File di build – pom.xml	51
6	Attività di Testing	52

6.1	Test plan	52
6.1.1	Funzionalità scelta e motivazione	52
6.1.2	Obiettivo e perimetro	52
6.1.3	Precondizioni e dati di prova	53
6.1.4	Casi di prova	54
6.1.5	Strategie di progettazione dei test	55
6.1.6	Matrice di tracciabilità	58
6.2	Test automatici di unità	59
6.2.1	Metodo 1 e 2: <code>newIssue(DT0, String)</code> e <code>handleIssue(int, String)</code>	59
6.2.2	Metodo 3: <code>getSuggestion(String, int)</code>	63
6.3	Strategie di progettazione dei test	64
6.3.1	Approccio	64
6.3.2	Casi limite e valori di confine	64
6.3.3	Copertura strutturale	65

Indice delle figure

1	Use Case Diagram #1 - [UC1,UC2]	18
2	Use Case Diagram #2 - [UC3,UC4,UC5]	20
3	Use Case Diagram #3 - [UC6,UC7,UC8]	21
4	Use Case Diagram #4 - [UC9]	21
5	Use Case Diagram #5 - [UC10]	22
6	Use Case Diagram #6 - [UC11, UC12]	23
7	Use Case Diagram #7 - [UC13]	23
8	Use Case Diagram #8 - Complessivo	24
9	Mock-up derivante dalla modellazione del caso d'uso UC9	25
10	Mock-up derivante dalla modellazione dei casi d'uso UC3,UC4,UC5	26
11	Vista generale dell'architettura del sistema	31
12	Diagramma UML concettuale per la persistenza dei dati	41
13	Diagramma UML ristrutturato per la persistenza dei dati	43
14	Diagramma delle classi del backend	46
15	Diagramma delle classi del frontend	47
16	Grafico della frequenza dei commit nel tempo	50
17	Report SonarQube	51

Introduzione

Il presente elaborato analizza e documenta nella sua interezza il sistema **BugBoard26**, sviluppato nell'ambito del corso di Ingegneria del Software A.A. 2025/2026.

BugBoard26 nasce come risposta all'esigenza di gestire in modo efficiente e collaborativo le issue all'interno di progetti software.

In contesti di sviluppo moderni, caratterizzati da team distribuiti e cicli di release sempre più rapidi, la capacità di tracciare, assegnare e monitorare problemi e richieste diventa fondamentale per il successo del progetto.

La piattaforma si propone di integrare funzionalità avanzate di issue tracking con un'interfaccia intuitiva e strumenti analitici, ponendo particolare attenzione all'esperienza utente e alla scalabilità del sistema.

L'architettura distribuita, che separa chiaramente backend e frontend, garantisce flessibilità nell'evoluzione del prodotto e nelle scelte tecnologiche.

Il documento che segue organizza i passi della progettazione in modo strutturato, facilitando la tracciabilità durante l'intero ciclo di vita del sviluppo software.

1 Glossario

1.1 Definizioni

Issue

Segnalazione di un problema, richiesta o domanda relativa al progetto software

Bug

Tipo di issue che rappresenta un malfunzionamento del software

Feature

Tipo di issue che rappresenta una richiesta di nuova funzionalità

Question

Tipo di issue che rappresenta una richiesta di chiarimento

Documentation

Tipo di issue relativa a problemi nella documentazione

Priorità

Livello di importanza assegnato a un issue

Stato

Condizione corrente di un issue (todo, in progress, done, etc.)

Amministratore

Utente con privilegi elevati che può gestire utenti e tutte le issue

Utente standard

Utente che può creare e modificare le issue a lui assegnate

Dashboard

Pannello di controllo che mostra informazioni aggregate

Tag

Marcatore personalizzabile associato ai bug

Stakeholder

Tutti i soggetti che, a vario titolo, interagiscono con il sistema o ne influenzano lo sviluppo e l'evoluzione.

Richiesta attività fornitori esterni

Documento che formalizza l'affidamento di attività, servizi o forniture a fornitori esterni, descrivendone oggetto, requisiti, tempi e modalità di approvazione.

1.2 Acronimi

SRS

Software Requirements Specification

RF Requisiti Funzionali

RNF

Requisiti Non Funzionali

UC Use Case - Caso d'uso

UCD

Use Case Diagram - Diagramma dei Casi d'uso

UML

Unified Modeling Language - Linguaggio di modellazione standard

ISO/IEC/IEEE

International Organization for Standardization / International Electrotechnical Commission / Institute of Electrical and Electronics Engineers

1.3 Elementi Informativi

1. **ISO/IEC/IEEE 29148:2018(E)** - Systems and software engineering - Life cycle processes - Requirements engineering
2. **Richiesta di attività fornitori esterni** - "Progetto di Ingegneria del Software - A.A. 2025/2026", SoftEngUniNA
3. **UML 2.5 Specification**

2 Descrizione generale

2.1 Prospettiva del prodotto

BugBoard26 rappresenta una soluzione completa per la gestione delle segnalazioni tecniche nell'ambito dello sviluppo software. Il sistema è progettato per facilitare la collaborazione tra i membri del team attraverso un'interfaccia intuitiva e funzionalità mirate all'ottimizzazione dei processi lavorativi.

Posizionandosi come strumento di supporto alle attività di sviluppo, il prodotto si caratterizza per la sua capacità di integrare tracciamento delle issue, comunicazione team e reporting in un'unica piattaforma coerente. L'approccio modulare garantisce scalabilità e adattabilità a diverse esigenze organizzative.

Destinato principalmente a team di sviluppo software e project manager, BugBoard26 si propone di migliorare l'efficienza operativa riducendo la complessità gestionale e promuovendo una collaborazione più strutturata tra i partecipanti ai progetti.

2.2 Fondamenti dei requisiti

2.2.1 Funzionalità del prodotto

Le funzionalità principali del sistema includono:

- **Gestione utenti:** Autenticazione, autorizzazione e gestione dei profili
- **Gestione issue:** Creazione, modifica, personalizzazione e tracciamento delle issue
- **Reporting:** Dashboard amministrative e report analitici
- **Ricerca e filtri:** Ricerca avanzata e filtraggio delle issue

2.2.2 Stakeholder

In base a ciò che si evince nel documento di **richiesta attività fornitori esterni**, sono state individuate le seguenti categorie principali di stakeholder:

- **Utenti Standard**
Sono i principali utilizzatori della piattaforma. Utilizzano BugBoard26 per gestire le issue, creare nuove segnalazioni di bug, monitorarne lo stato e aggiungere allegati. Hanno un ruolo operativo e sono interessati alla semplicità d'uso, alla rapidità di consultazione e alla chiarezza dell'interfaccia.
- **Utenti Amministratori**
Hanno il compito di supervisionare l'avanzamento dei lavori, verificare lo stato delle issue e generare report di sintesi tramite la dashboard. Sono interessati a funzionalità di analisi e controllo, in particolare alla tracciabilità delle attività e all'affidabilità dei dati visualizzati. Gestiscono la piattaforma da un punto di vista tecnico: configurazione, creazione e gestione degli account, controllo dei permessi.
- **Committenza**
È l'organizzazione o il gruppo che ha commissionato lo sviluppo del sistema BugBoard26: SoftEngUniNA. Definisce i requisiti iniziali, approva le specifiche e valuta il rispetto degli obiettivi funzionali e non funzionali stabiliti nel documento.

- **Team di manutenzione**

È il gruppo di esperti che manutiene il sistema. Garantisce che la piattaforma sia correttamente funzionante durante il suo ciclo di vita.

2.2.3 Vincoli

Vincoli architetturali

- backend e frontend devono essere componenti indipendenti
- Il frontend deve poter essere sostituito senza modifiche al backend

Vincoli tecnologici

- Utilizzo obbligatorio di linguaggi Object-Oriented
- Utilizzo di tool CASE per la progettazione
- Persistenza dati gestita esclusivamente dal backend

Vincoli di sicurezza

- Autenticazione obbligatoria per accedere al sistema
- Gestione sicura delle credenziali
- Controllo degli accessi basato sui ruoli
- Preservazione dell'integrità e segretezza dei dati

3 Specifica dei Requisiti Software

La seguente specifica è stata redatta in conformità con lo standard ISO/IEC/IEEE 29148:2018(E) per garantire una definizione completa, coerente e verificabile dei requisiti.

3.1 Trasformazione dei bisogni in requisiti

In seguito alla precedente sezione in cui vengono definiti i **fondamenti dei requisiti** è possibile trasformare i bisogni descritti nella traccia di richiesta attività in requisiti. Quindi, la seguente tabella mostra come i principali bisogni degli stakeholder siano stati tradotti in requisiti del sistema **BugBoard26**:

Bisogno	Requisito
Gli utenti devono poter segnalare bug e richieste in modo semplice	RF-004 (Creazione issue), RF-005 (Allegati e priorità), RF-006 (Tipologie issue)
L'amministratore deve avere una panoramica delle attività e generare report	RF-011 (Dashboard amministratore), RF-012 (Metriche dashboard), RF-013 (Report mensili), RNF-001 / RNF-002 (Preselezioni)
Un amministratore deve creare e gestire i profili utenti	RF-001 (Login utente), RF-002 (Account amministratore predefinito), RF-003 (Creazione utenti)
Deve essere possibile cercare, filtrare e ordinare le issue in base a criteri	RF-008 (Vista riepilogativa), RF-009 (Filtraggio issue), RF-010 (Ordinamento issue)
Personalizzazione accessibile e semplice degli issue	RF-007 (Etichette personalizzabili), RNF-005 / RNF-006 (Usabilità)
Il sistema deve essere affidabile, sicuro e manutenibile nel tempo	RNF-003 (Disponibilità sistema), RNF-004 (Integrità dati), RNF-005 / RNF-006 (Usabilità), RNF-007 (Sicurezza), RNF-008 / RNF-009 (Manutenibilità), RNF-010 / RNF-011 (Portabilità)
Gli utenti autenticati devono poter accedere al sistema in modo sicuro	RF-001 (Login utente), RNF-007 (Hashing password)
Le funzionalità principali devono essere facilmente accessibili e intuitive	RNF-005 (Interfaccia intuitiva), RNF-006 (Accessibilità funzionalità principali)
Il sistema deve operare con tempi di risposta accettabili	RNF-001 (Tempo di risposta operazioni comuni), RNF-002 (Ricerca in tempo reale)
Il sistema deve mantenere l'indipendenza tra componenti frontend e backend	RNF-010 (Indipendenza backend/frontend), RNF-011 (Containerizzazione)
Il codice deve essere ben documentato e manutenibile	RNF-008 (Documentazione codice), RNF-009 (Architettura estensibile)

Tabella 1: Mappatura bisogni - requisiti del sistema

3.2 Attori del sistema

Di seguito si descrivono i ruoli degli attori del sistema e le loro responsabilità, in conformità alle richieste del committente.

Utente

Descrizione: Membro del team di sviluppo

Responsabilità: Gestione delle issue

Privilegi:

- Creazione di nuove issue
- Modifica delle issue assegnate
- Visualizzazione di tutte le issue
- Ricerca e filtraggio issue

Scenario: Segnalazione problemi, aggiornamento stato issue assegnate, collaborazione su issue di team

Amministratore

Descrizione: *Specializzazione* dell'Utente con privilegi di gestione del sistema

Responsabilità: Gestione utenti, configurazione sistema, accesso a tutte le funzionalità

Privilegi:

- Creazione e gestione di tutti gli utenti
- Accesso alla dashboard amministrativa

Scenario: Monitoraggio attività del team, generazione report

Team

Descrizione: Team di sviluppo composto da Utenti della piattaforma

Responsabilità: Sviluppo software collaborativo facilitato dal sistema stesso

Scenario: Gestione e collaborazione collettiva degli issue

Persona non registrata

Descrizione: Persona selezionata da un Amministratore per cui si verifica successivamente la creazione di un proprio account personale nel sistema

Responsabilità: Futuro Utente della piattaforma

Scenario: Nuovo membro del Team di sviluppo

3.3 Requisiti Specifici

Di seguito si descrive in dettaglio i requisiti funzionali e non funzionali del sistema.

3.3.1 Requisiti Funzionali

RF01 - Autenticazione e gestione utenti

Identificatore: RF-001

Titolo: Login utente

Descrizione: Il sistema deve permettere il login tramite email e password

Priorità: Alta

Fonte: Traccia progetto, punto 3.1.1

Criteri di accettazione: L'utente può accedere con credenziali valide, accesso negato con credenziali errate

Identificatore: RF-002

Titolo: Account amministratore predefinito

Descrizione: Il sistema deve essere fornito con un account amministratore già attivo con credenziali di default

Priorità: Alta

Fonte: Traccia progetto, punto 3.1.1

Criteri di accettazione: Possibile effettuare login con credenziali amministrative predefinite

Identificatore: RF-003

Titolo: Creazione nuovi utenti

Descrizione: L'amministratore deve poter creare nuovi utenti specificando email, password e tipo (normale/amministrazione)

Priorità: Alta

Fonte: Traccia progetto, punto 3.1.1

Criteri di accettazione: Nuovo utente creato può effettuare il login con le credenziali specificate

RF02 - Creazione e personalizzazione issue

Identificatore: RF-004

Titolo: Creazione issue

Descrizione: Gli utenti autenticati devono poter creare issue indicando almeno titolo e descrizione

Priorità: Alta

Fonte: Traccia progetto, punto 3.1.2

Criteri di accettazione: Issue creata appare nella lista con stato "todo"

Identificatore: RF-005

Titolo: Specifica opzionale priorità e allegati

Descrizione: Il sistema deve permettere di specificare opzionalmente priorità e allegare immagini alle issue

Priorità: Media

Fonte: Traccia progetto, punto 3.1.2

Criteri di accettazione: Issue creata con priorità e/o immagine allegata mostra correttamente questi dati

Identificatore: RF-006

Titolo: Tipologie di issue

Descrizione: Le issue devono essere classificabili in: question, bug, documentation, feature

Priorità: Alta

Fonte: Traccia progetto, punto 3.1.2

Criteri di accettazione: Ogni issue creata ha una tipologia assegnata correttamente

Identificatore: RF-007

Titolo: Etichette personalizzabili

Descrizione: Il sistema deve permettere di associare un numero variabile di etichette personalizzabili alle issue di tipo "bug"

Priorità: Media

Fonte: Traccia progetto, punto 3.1.10

Criteri di accettazione: Bug può avere multiple etichette assegnate

RF03 - Visualizzazione e ricerca issue

Identificatore: RF-008

Titolo: Vista riepilogativa issue

Descrizione: Il sistema deve offrire una vista riepilogativa di tutte le issue

Priorità: Alta

Fonte: Traccia progetto, punto 3.1.3

Criteri di accettazione: Tutte le issue sono visualizzabili in una vista organizzata

Identificatore: RF-009

Titolo: Filtraggio issue

Descrizione: Deve essere possibile filtrare le issue per tipologia, stato, priorità e altri parametri

Priorità: Media

Fonte: Traccia progetto, punto 3.1.3

Criteri di accettazione: Le issue possono essere filtrate secondo i criteri specificati

Identificatore: RF-010

Titolo: Ordinamento issue

Descrizione: Deve essere possibile ordinare le issue per tipologia, stato, priorità e altri parametri

Priorità: Media

Fonte: Traccia progetto, punto 3.1.3

Criteri di accettazione: Le issue possono essere ordinate secondo i criteri specificati

RF04 - Dashboard e reporting

Identificatore: RF-011

Titolo: Dashboard amministratore

Descrizione: Gli amministratori devono avere accesso a una dashboard che mostri informazioni aggregate

Priorità: Alta

Fonte: Traccia progetto, punto 3.1.7

Criteri di accettazione: Dashboard accessibile e mostra dati aggregati

Identificatore: RF-012

Titolo: Metriche dashboard

Descrizione: La dashboard deve mostrare: numero di issue aperte, tempo medio di risoluzione

Priorità: Media

Fonte: Traccia progetto, punto 3.1.7

Criteri di accettazione: Tutte le metriche specificate sono visualizzate nella dashboard

Identificatore: RF-013

Titolo: Report mensili

Descrizione: Gli amministratori devono poter visualizzare report mensili sull'attività del team con metriche aggregate e dettagliate per utente

Priorità: Bassa

Fonte: Traccia progetto, punto 3.1.17

Criteri di accettazione: Report mensili generabili e mostrano metriche richieste

Identificatore: RF-014

Titolo: Suggerimenti automatici

Descrizione: Il sistema deve suggerire automaticamente a un utente di risolvere un bug, sulla base del suo carico di lavoro attuale

Priorità: Media

Fonte: Traccia progetto, punto 3.1.14

Criteri di accettazione: Gli utenti con meno carico ricevono il suggerimento

3.3.2 Requisiti Non Funzionali

RNF01 - Prestazioni

Identificatore: RNF-001

Titolo: Tempo di risposta operazioni comuni

Descrizione: Le operazioni comuni (login, visualizzazione liste, creazione issue) devono completarsi in tempi di risposta accettabili

Priorità: Alta

Tipo: Prestazione

Criteri di accettazione: Test di performance confermano tempi di risposta

Identificatore: RNF-002

Titolo: Ricerca in tempo reale

Descrizione: La ricerca delle issue deve restituire risultati in tempo reale

Priorità: Media

Tipo: Prestazione

Criteri di accettazione: Ricerche completano entro 1 secondo

RNF02 - Affidabilità

Identificatore: RNF-003

Titolo: Disponibilità sistema

Descrizione: Il sistema deve garantire un uptime del 99% durante gli orari lavorativi

Priorità: Alta

Tipo: Affidabilità

Metrica: Uptime $\geq 99\%$

Criteri di accettazione: Monitoraggio conferma disponibilità

Identificatore: RNF-004

Titolo: Integrità dati

Descrizione: Il sistema deve garantire l'integrità dei dati in caso di failure

Priorità: Alta

Tipo: Affidabilità

Metrica: Nessuna perdita dati in condizioni normali

Criteri di accettazione: Test di recovery confermano integrità dati

RNF03 - Usabilità

Identificatore: RNF-005

Titolo: Interfaccia intuitiva

Descrizione: L'interfaccia deve essere utilizzabile senza training formale per utenti con esperienza in sistemi di issue tracking

Priorità: Alta

Tipo: Usabilità

Metrica: Task completion rate elevata per tutti i tipi di utilizzatori

Criteri di accettazione: Test utilizzatore confermano facilità d'uso

Identificatore: RNF-006

Titolo: Accessibilità funzionalità principali

Descrizione: Le funzionalità principali devono essere accessibili facilmente

Priorità: Media

Tipo: Usabilità

Criteri di accettazione: Analisi navigazione conferma accessibilità

RNF04 - Sicurezza

Identificatore: RNF-007

Titolo: Hashing password

Descrizione: Le password devono essere memorizzate in forma hashata utilizzando algoritmi sicuri.

Priorità: Alta

Tipo: Sicurezza

Metrica: Password non memorizzate in chiaro

Criteri di accettazione: Analisi di sicurezza conferma assenza leakage

RNF05 - Manutenibilità

Identificatore: RNF-008

Titolo: Documentazione software

Descrizione: Il software deve essere ben documentato in tutte le sue fasi, per garantire chiarezza e trasparenza

Priorità: Media

Tipo: Manutenibilità

Metrica: Presenza documento per ogni artefatto prodotto

Criteri di accettazione: Documenti prodotti adempiono al loro scopo di descrizione correttamente

Identificatore: RNF-009

Titolo: Architettura estendibile

Descrizione: L'architettura deve permettere l'aggiunta di nuove funzionalità senza modifiche massive alla stessa

Priorità: Media

Tipo: Manutenibilità

Metrica: Modularità e basso accoppiamento

Criteri di accettazione: Design review conferma estendibilità

RNF06 - Portabilità

Identificatore: RNF-010

Titolo: Indipendenza backend/frontend

Descrizione: Il backend deve essere completamente indipendente dal frontend e viceversa

Priorità: Alta

Tipo: Portabilità

Metrica: frontend sostituibile senza modifiche backend

Criteri di accettazione: Test di sostituzione componenti confermano indipendenza

Identificatore: RNF-011

Titolo: Containerizzazione

Descrizione: Utilizzo di tecnologie di containerizzazione per il deployment

Priorità: Bassa

Tipo: Portabilità

Criteri di accettazione: Software containerizzato disponibile all'uso

3.4 Modellazione dei casi d'uso

In questa sezione vengono presentati i casi d'uso principali della piattaforma.

3.4.1 Descrizione casi d'uso principali

UC1 - Accedi alla piattaforma

Attori: Utente

Precondizioni: L'utente possiede credenziali valide

Flusso principale:

1. L'utente accede alla pagina di login
2. L'utente inserisce email e password
3. Il sistema verifica le credenziali
4. Il sistema autentica l'utente e reindirizza alla home page

Postcondizioni: L'utente è autenticato e può accedere alle funzionalità

UC2 - Crea nuova utenza

Attori: Amministratore, persona non registrata

Precondizioni: L'amministratore è autenticato correttamente

Flusso principale:

1. L'amministratore definisce le credenziali del nuovo utente e le inserisce nella piattaforma
2. Il sistema crea la nuova utenza
3. L'utente riceve le credenziali appena convalidate

Postcondizioni: L'utente può accedere alla piattaforma correttamente

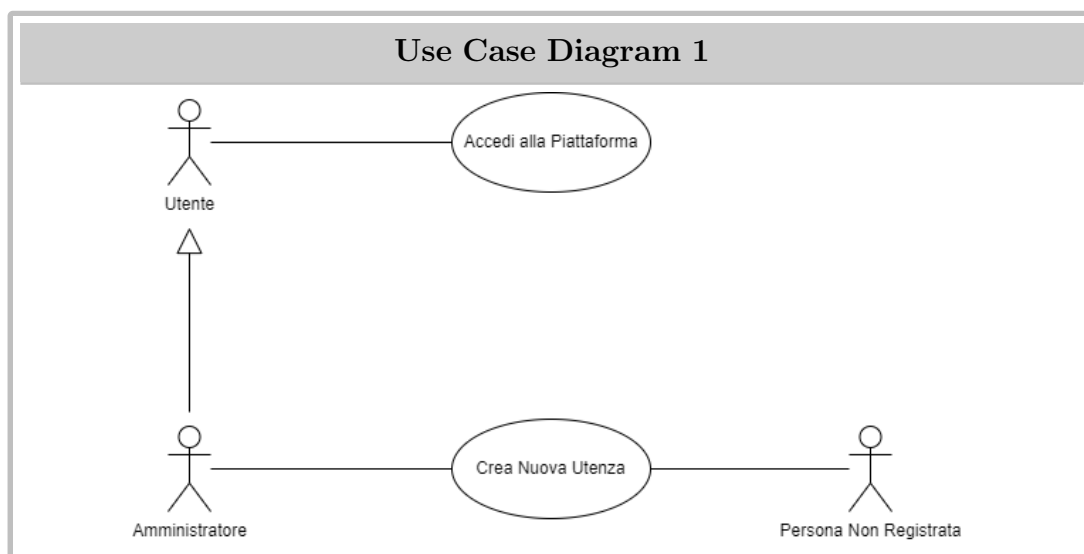


Figura 1: Use Case Diagram #1 - [UC1,UC2]

UC3 - Segnala un issue

Attori: Utente

Precondizioni: L'utente è autenticato

Flusso principale:

1. L'utente accede alla funzione di creazione issue
2. L'utente inserisce titolo e descrizione
3. l'utente specifica tipo dell'issue
4. opzionalmente specifica priorità e allega un'immagine
5. L'utente conferma la creazione
6. Il sistema crea l'issue con stato "todo"

Postcondizioni: L'issue è creata e visibile nel sistema

UC4 - Specifica una priorità

Attori: Utente

Precondizioni: L'utente sta svolgendo il caso d'uso **Segnala un issue** **Flusso principale:**

1. L'utente sceglie un livello di priorità dell'issue consono
2. La piattaforma assegna il livello di priorità scelto all'issue

Postcondizioni: L'issue presenta la priorità correttamente

UC5 - Aggiungi un'immagine

Attori: Utente

Precondizioni: L'utente sta svolgendo il caso d'uso **Segnala un issue**

Flusso principale:

1. L'utente sceglie l'immagine da allegare all'issue
2. La piattaforma associa correttamente l'immagine all'issue

Postcondizioni: La piattaforma consente la visualizzazione dell'immagine

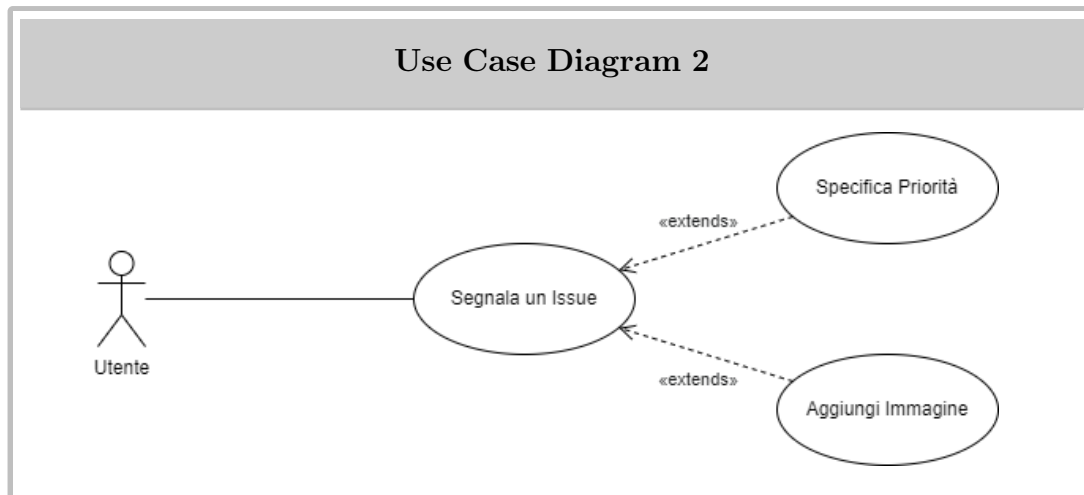


Figura 2: Use Case Diagram #2 - [UC3,UC4,UC5]

UC6 - Visualizza riepilogo

Attori: Utente

Precondizioni: L'utente è autenticato

Flusso principale:

1. L'utente accede alla sezione di riepilogo issue
2. Il sistema seleziona gli issue disponibili per essere mostrati
3. L'utente visualizza i dati correttamente

UC7 - Filtra risultati

Attori: Utente

Precondizioni: L'utente sta svolgendo il caso d'uso Visualizza riepilogo

Flusso principale:

1. L'utente definisce i parametri del filtro
2. La piattaforma estrae correttamente i dati da mostrare

UC8 - Ordina risultati

Attori: Utente

Precondizioni: L'utente sta svolgendo il caso d'uso Visualizza riepilogo

Flusso principale:

1. L'utente definisce i parametri dell'ordinamento
2. La piattaforma riordina correttamente i dati da mostrare

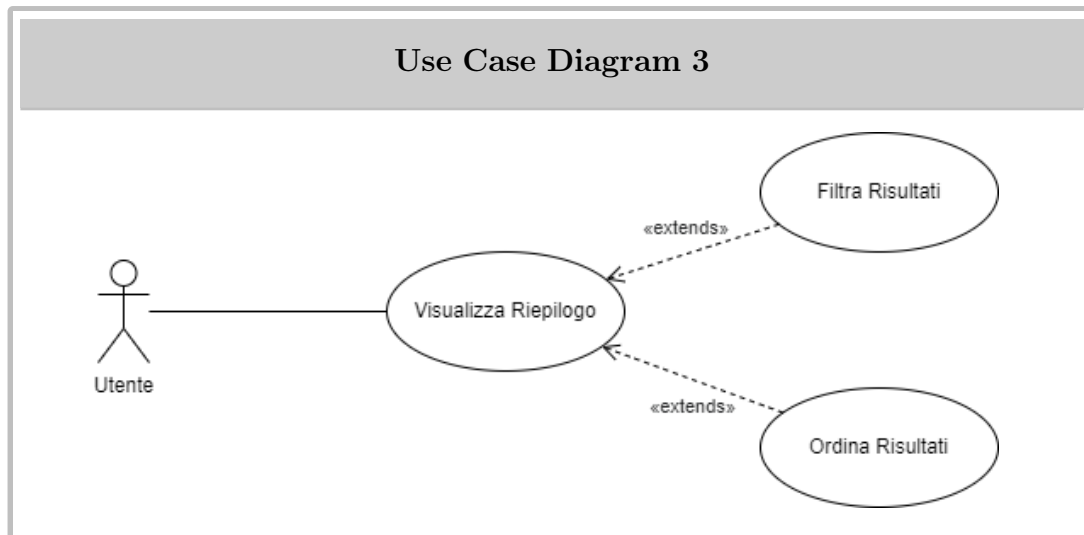


Figura 3: Use Case Diagram #3 - [UC6,UC7,UC8]

UC9 - Visualizza dashboard

Attori: Amministratore

Precondizioni: L'amministrazione è autenticato

Flusso principale:

1. L'amministratore accede alla pagina di resoconto
2. La piattaforma restituisce i dati operativi richiesti in modo coerente

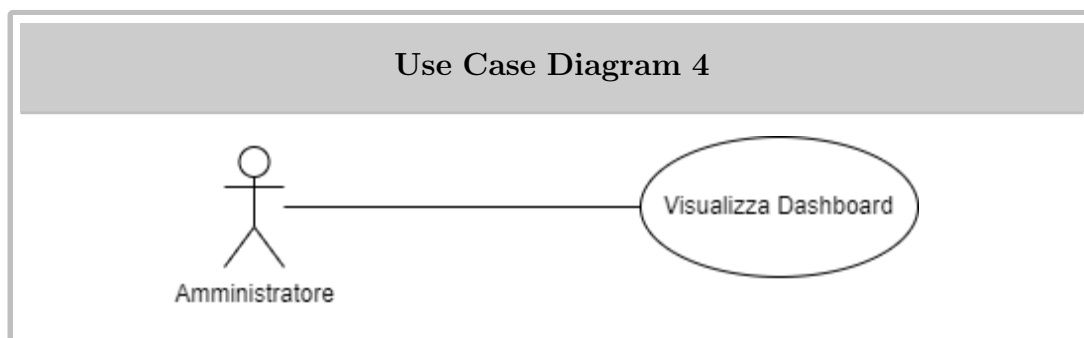


Figura 4: Use Case Diagram #4 - [UC9]

UC10 - Aggiungi etichetta

Attori: Utente

Precondizioni: L'utente è autenticato e ha selezionato un issue

Flusso principale:

1. L'utente determina una o più etichette appropriate e le appone all'issue scelto
2. Il sistema memorizza la combinazione di tag e la relativa assegnazione

Postcondizioni: La piattaforma salva con successo l'aggiunta apportata dall'utente

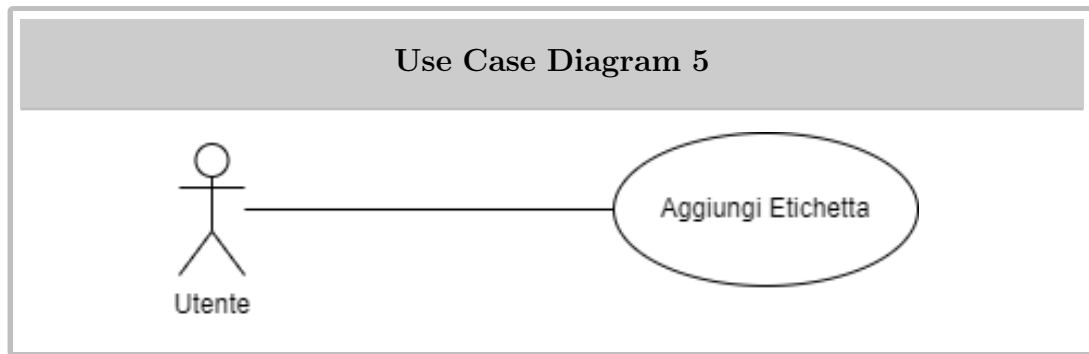


Figura 5: Use Case Diagram #5 - [UC10]

UC11 - Svolgi attività

Attori: Team

Precondizioni: Nessuna

Flusso principale:

1. Il team di sviluppo svolge diverse attività di risoluzione issue
2. La piattaforma tiene traccia correttamente delle evoluzioni del software

UC12 - Visualizza report mensile

Attori: Amministratore

Precondizioni: Almeno un Team ha svolto il caso d'uso Svolgi attività

Flusso principale:

1. L'amministratore seleziona il team di cui visualizzare il report
2. L'amministratore accede alla funzione di visualizzazione report
3. Il sistema mostra il report mensile delle attività del team

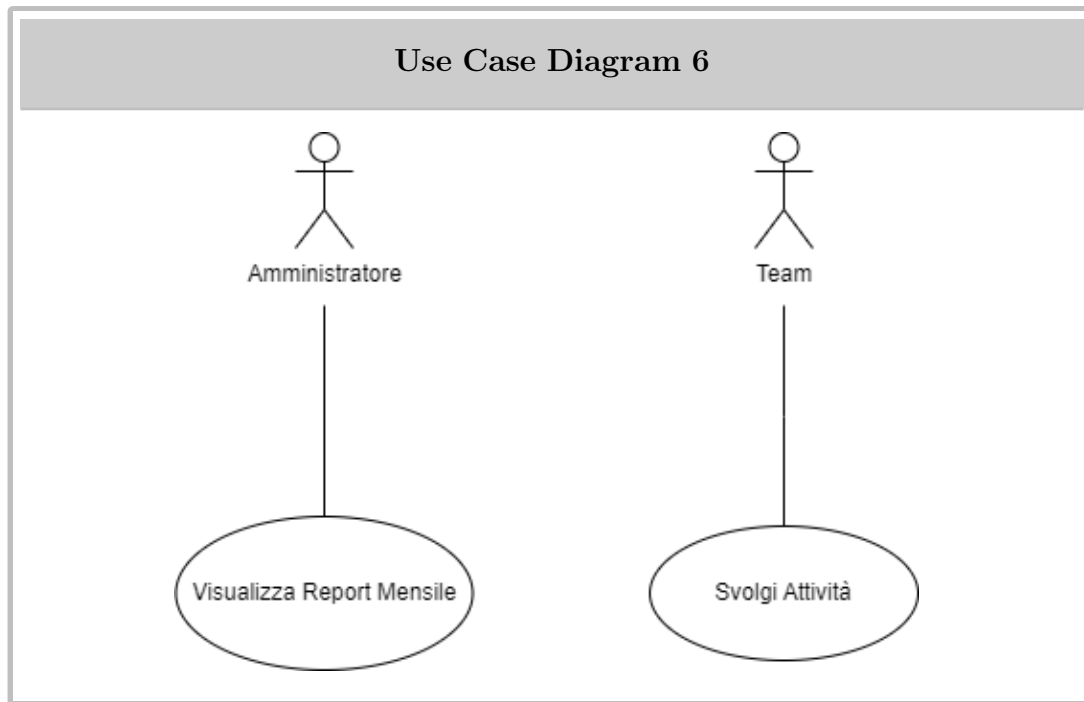


Figura 6: Use Case Diagram #6 - [UC11, UC12]

UC13 - Accetta suggerimento automatico

Attori: Utente

Precondizioni: L'utente è autenticato e ha un basso carico di lavoro

Flusso principale:

1. Il sistema calcola la distribuzione del lavoro e seleziona l'utente
2. Il sistema notifica l'utente e lo invita a scegliere un bug
3. L'utente seleziona un bug su cui lavorare

Postcondizioni: Il bug risulta correttamente preso in carica dall'utente

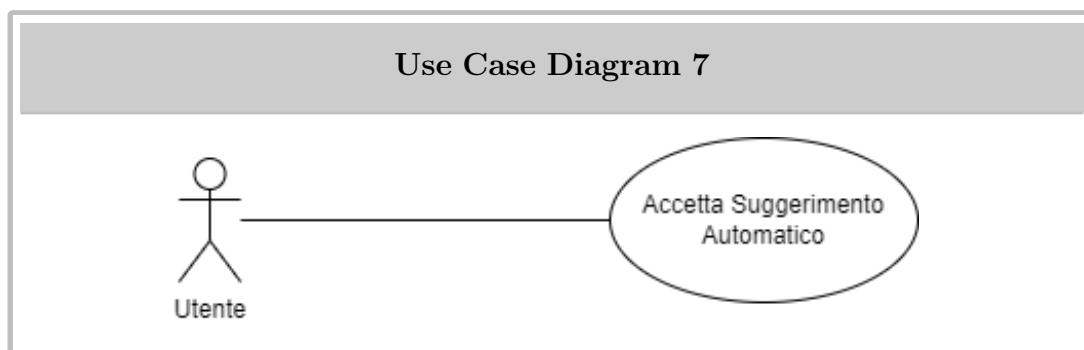


Figura 7: Use Case Diagram #7 - [UC13]

Use Case Diagram Complessivo

Il presente *Use Case Diagram* (UCD) rappresenta una visione d'insieme delle feature richieste dal committente, modellate secondo i formalismi descritti dall'Unified Modeling Language

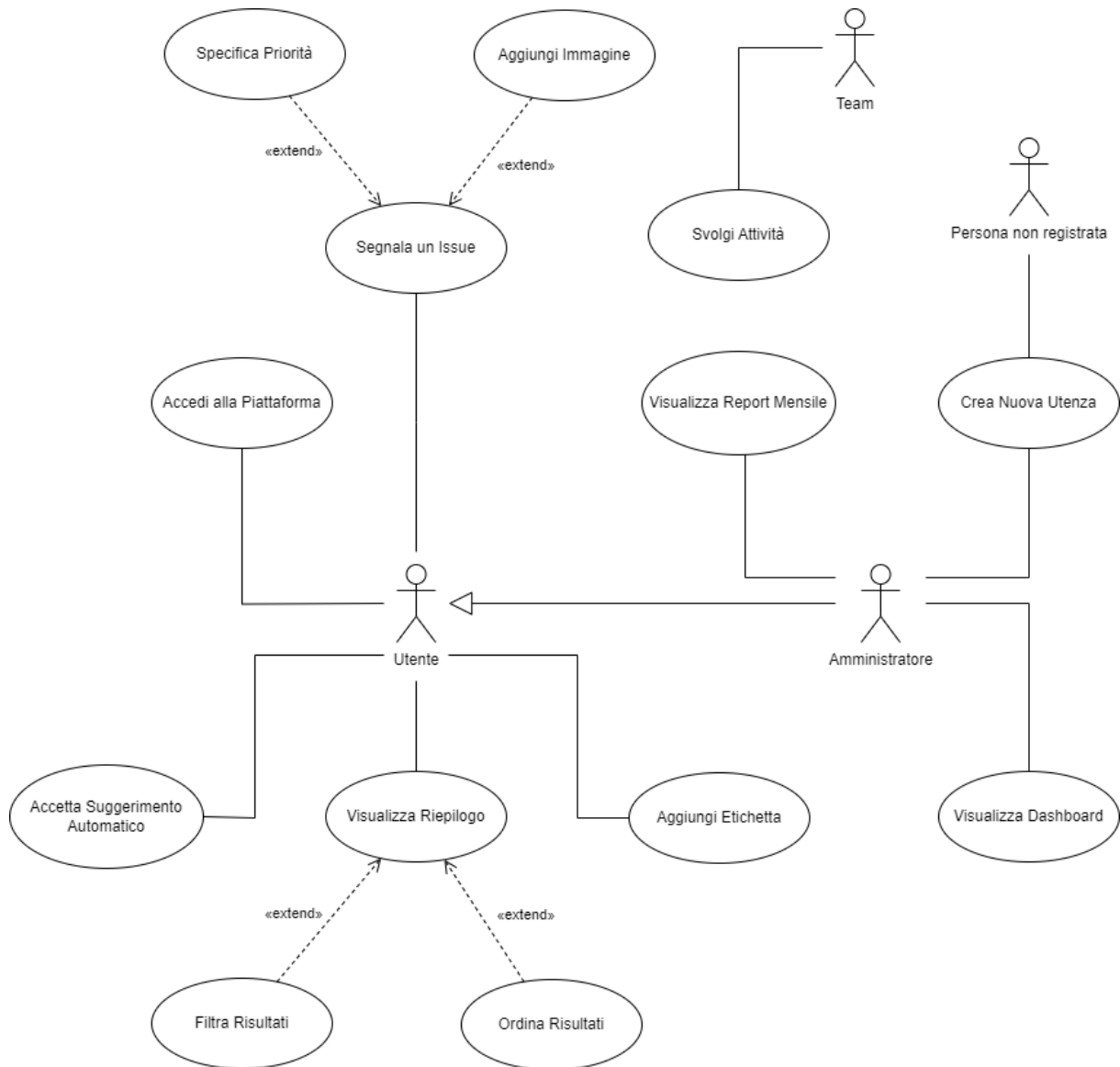


Figura 8: Use Case Diagram #8 - Complessivo

3.5 Modellazione di casi d'uso

Di seguito la modellazione estesa degli Use Case menzionati tramite **mock-up** e **formalismo di Cockburn**:

- Visualizza dashboard
(UC9)
- Segnala un Issue
con relative estensioni Specifica Priorità e Aggiungi Immagine
(UC3, UC4, UC5)

3.5.1 Mock-ups

Visualizza dashboard

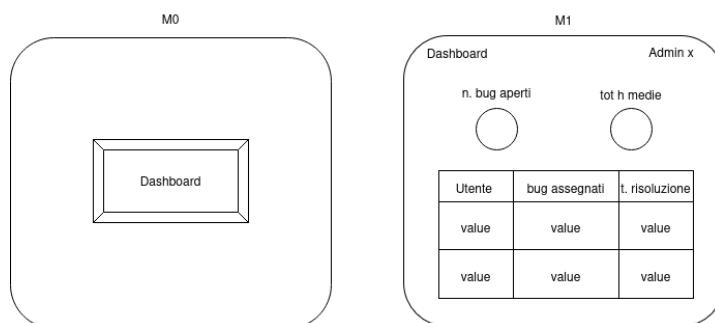


Figura 9: Mock-up derivante dalla modellazione del caso d'uso UC9

Segnala un Issue

M0

Segnala un Issue

M1

Segnala un issue

Titolo:

Descrizione:

Priorità

Immagine

Segnala

M2

L'issue è stato segnalato correttamente!

Ok

M3

Alta

Media

Bassa

Ok

M4

Scegli allegato:

Sfoggia

Ok

M5

Compila tutti i campi!

Ok

M6

Seleziona un'opzione o annulla!

Ok

M7

Seleziona un'immagine o annulla!

Ok

M8

La dimensione è superiore al limite! (100Mb)

Riprova

Ok

M9

Questo formato non è supportato! (.png, .jpg, .jpeg)

Riprova

Ok

Figura 10: Mock-up derivante dalla modellazione dei casi d'uso UC3,UC4,UC5

3.5.2 Formalismi di Cockburn

Visualizza dashboard

USE CASE #9 - Visualizza dashboard		
Goal in Context	L'amministratore intende visualizzare informazioni aggregate su bug e utenti	
Preconditions	L'utente è autenticato correttamente e risulta un admin della piattaforma	
Success End Condition	L'admin visualizza correttamente la dashboard	
Failed End Condition	L'admin non visualizza la dashboard	
Primary Actor	Admin	
Trigger	L'admin all'interno della sua area riservata preme il pulsante "Dashboard"	
MAIN SCENARIO		
Step n.	Admin	Sistema
1	Clicca "Dashboard" in M0	
2		Mostra M1 – Visualizzazione Dashboard

Segnala un Issue

USE CASE #3 - Segnala un issue		
Goal in Context	Un utente vuole segnalare un issue sulla piattaforma	
Preconditions	L'utente è autenticato correttamente	
Success End Condition	L'issue è creato correttamente, memorizzato dalla piattaforma e disponibile per la risoluzione	
Failed End Condition	L'issue non viene pubblicato sulla piattaforma	
Primary Actor	Utente	
Trigger	L'utente accede alla schermata per la segnalazione di un issue e clicca sul bottone di “Segnala un issue” in M0	
MAIN SCENARIO		
Step n.	Utente	Sistema
1	Clicca “Segnala un issue” in M0	
2		Mostra M1 – Creazione dell'issue
3	Compila i campi	
4	Clicca “Segnala”	
5		Mostra M2 – L'issue è stato segnalato
6	Clicca “Ok”	
7		Chiude la finestra

EXTENSION #1: Specifica Priorità		
4a	Clicca “Priorità”	
5a		Mostra M3 – Selezione della priorità
6a	Seleziona un’opzione e clicca “Ok”	
7a		Mostra M1 – Ritorna allo step 3 del Main Scenario
EXTENSION #2: Aggiungi immagine		
4b	Clicca “Immagine”	
5b		Mostra M4 – Scelta dell’allegato
6b	Clicca “Sfoglia”	
7b		Mostra selettore file di sistema
8b	Seleziona file	
9b		Torna a M4
10b	Clicca “Ok”	
11b		Mostra M1 – Ritorna allo step 3 del Main Scenario
EXTENSION #3: Errore di mancata compilazione dei campi		
3c	Clicca “Segnala” con uno o più campi vuoti	
4c		Mostra M5 – Errore sulla completezza dei campi
5c	Clicca “Ok”	
6c		Mostra M1 – Ritorna allo step 3 del Main Scenario
EXTENSION #1.1: Errore di mancata scelta della priorità		
6aa	Clicca “Ok” con scelta nulla	
7aa		Mostra M6 – Opzione nulla
8aa	Clicca “Ok”	
9aa		Mostra M3 – Ritorna allo step 6a dell’Extension #1
EXTENSION #2.1: Errore di mancata selezione d’immagine		
6ba	Clicca “Ok” senza aver selezionato un file	
7ba		Mostra M7 – Immagine nulla
8ba	Clicca “Ok”	
9ba		Mostra M4 – Ritorna allo step 6b dell’Extension #2

EXTENSION #2.2: Errore relativo la dimensione dell'immagine		
11bb		Mostra M8 – Immagine di dimensioni troppo grandi
12bb	Clicca “Ok”	
13bb		Mostra M4 – Ritorna allo step 6b dell'Extension #2
EXTENSION #2.3: Errore formato dell'immagine non supportato		
11bc		Mostra M9 – Formato dell'immagine non supportato
12bc	Clicca “Ok”	
13bc		Mostra M4 – Ritorna allo step 6b dell'Extension #2

3.6 Tracciamento dei requisiti

3.6.1 Tracciamento fonte-requisito

Fonte	Descrizione	Requisiti
Traccia 3.1.1	Sistema autenticazione	RF-001, RF-002, RF-003
Traccia 3.1.2	Creazione issue	RF-004, RF-005, RF-006
Traccia 3.1.3	Vista riepilogativa e ricerca	RF-008, RF-009, RF-010
Traccia 3.1.7	Dashboard amministratore	RF-011, RF-012
Traccia 3.1.10	Etichette personalizzabili	RF-007
Traccia 3.1.17	Report mensili	RF-013

Tabella 4: Tracciamento requisiti dalla traccia di progetto

3.6.2 Tracciamento requisiti-casi d'uso

Requisito	Casi d'uso	Descrizione
RF-001	UC1	Login utente
RF-002	UC1	Account amministratore predefinito
RF-003	UC2	Creazione nuovi utenti
RF-004	UC3	Creazione issue
RF-005	UC4, UC5	Specifica priorità e allegati
RF-006	UC3	Tipologie di issue
RF-007	UC10	Etichette personalizzabili
RF-008	UC6	Vista riepilogativa issue
RF-009	UC7	Filtraggio issue
RF-010	UC8	Ordinamento issue
RF-011	UC9	Dashboard amministratore
RF-012	UC9	Metriche dashboard
RF-013	UC11, UC12	Report mensili

Tabella 5: Tracciamento requisiti funzionali - casi d'uso

4 Design del Sistema

4.1 Descrizione Architettuale

4.2 Panoramica dell'architettura

BugBoard26 adotta un'architettura client-server a due livelli, in cui frontend e backend sono processi completamente separati che comunicano esclusivamente tramite API REST su HTTP/JSON (si veda RNF-010). Il backend è un'applicazione Spring Boot che espone gli endpoint REST, applica le regole di autorizzazione (JWT + ruoli) e gestisce l'unica fonte di verità dei dati, un database PostgreSQL ospitato su Neon (Postgres serverless in cloud, Sezione 4.9). Il frontend è un'applicazione desktop Java Swing che gestisce la presentazione dei dati e le interazioni con l'utente. Può eseguire controlli locali sui campi inseriti per fornire un riscontro immediato; le validazioni definitive, le regole di business e l'autorizzazione delle operazioni sono applicate dal backend. Il frontend invoca gli endpoint REST e presenta all'utente i risultati ricevuti.

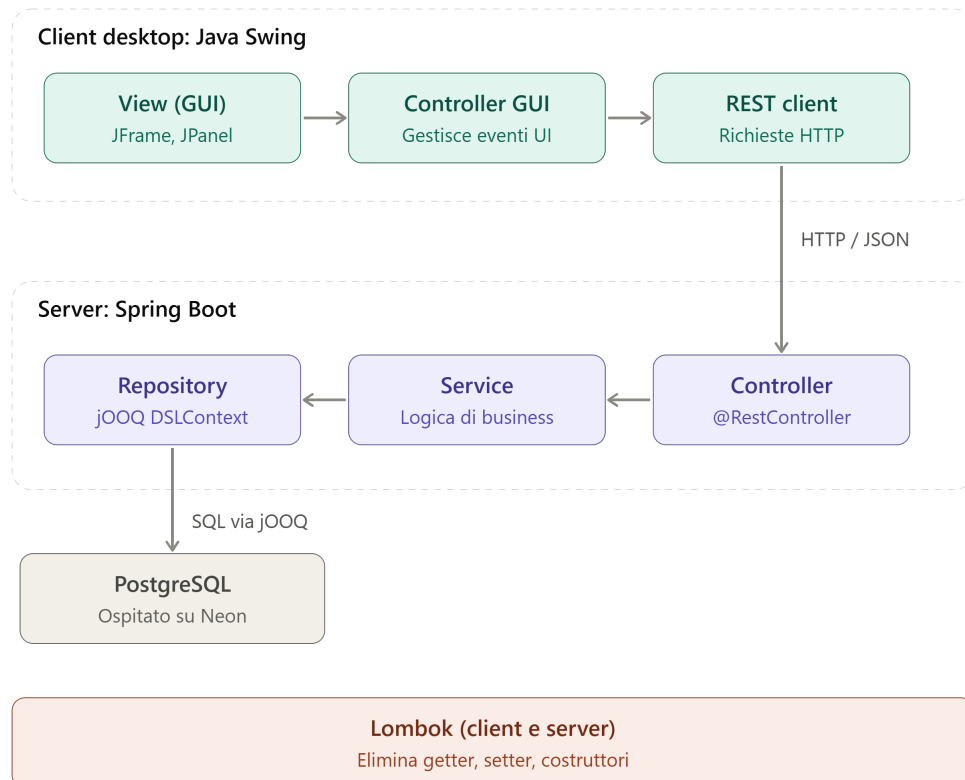


Figura 11: Vista generale dell'architettura del sistema

4.3 Criteri di design adottati

4.3.1 Criteri funzionali

Le scelte architetturali sono state guidate principalmente da:

- **supporto diretto ai casi d'uso della Sezione 3.6:** ogni endpoint REST del backend corrisponde a uno o più casi d'uso (login, creazione/gestione issue, dashboard, report, gestione progetti/team);
- **indipendenza e sostituibilità** (RNF-010): nessuna classe di dominio del backend (User, Issue, Team) è nota al frontend, che dipende solo dal contratto JSON;
- **estensibilità** (RNF-009): l'aggiunta di un nuovo endpoint richiede solo un nuovo metodo in un `@RestController` e, lato frontend, un nuovo metodo in un controller di dominio, senza toccare il resto del sistema.

4.3.2 Criteri di qualità

Attributo	Priorità	Motivazione
Sicurezza	Alta	Autenticazione obbligatoria (RNF-007), token JWT firmato, hashing BCrypt delle password, autorizzazione basata sui ruoli lato backend
Manutenibilità	Alta	Separazione netta backend/frontend (RNF-010), package organizzati per responsabilità, DTO dedicati a ogni scambio dati (RNF-008/009)
Portabilità	Alta	Backend e frontend indipendenti (RNF-010); persistenza su Postgres gestito (Neon) anziché istanza locale, per rendere il deployment del backend indipendente dalla macchina di sviluppo
Usabilità	Media	Interfaccia Swing pensata per essere utilizzabile senza training (RNF-005/006)
Performance	Media	Garanzia di rapidità delle operazioni per l'assenza di orchestratori complessi (RNF-001/002)

Tabella 6: Quality attributes e priorità

4.3.3 Vincoli architetturali

Le scelte di design sono vincolate da quanto specificato in Sezione 2.2.3: backend e frontend devono restare componenti indipendenti, sostituibili singolarmente. Questo si traduce concretamente in due regole di progetto rispettate in tutto il codice:

- nessuna classe del backend è importata o referenziata dal frontend, e viceversa;
- l'unico contratto condiviso tra i due moduli è il formato JSON scambiato via HTTP, documentato in Sezione 4.12.

A questi si aggiunge il vincolo tecnologico (linguaggi Object-Oriented) e quello di sicurezza di Sezione 3.2.3 (autenticazione obbligatoria, gestione sicura delle credenziali), entrambi rispettati rispettivamente dalla scelta di Java per entrambi i moduli e dall'accoppiata JWT + BCrypt lato backend (Sezione 4.11).

4.4 Pattern architetturale adottato

4.4.1 Descrizione del pattern

A livello di sistema è adottato un pattern **client-server**: il frontend Swing (client) e il backend Spring Boot (server) sono processi separati, comunicanti esclusivamente via REST/JSON. All'interno del backend, la logica è organizzata secondo un'**architettura a strati** leggera: un livello di **@RestController** (mapping HTTP, validazione input con **@Valid**, gestione errori), un livello di **Service** che si occupa di gestire la business logic complessa, un livello di **DAO** (interfacce che dichiarano le operazioni di dominio), e un livello di implementazione **jOOQ** che traduce le operazioni DAO in query SQL verso Postgres. È bene notare che il livello **Service** presenta solo le operazioni di logica di business concrete, in quanto rispettare la formalità dell'architettura per un software semplice avrebbe introdotto solo *bloat*, non utile alle reali operazioni CRUD.

4.4.2 Motivazione della scelta

Il pattern client-server a due componenti è imposto direttamente dal vincolo architetturale di indipendenza backend/frontend; tra le alternative valutate, un'architettura a microservizi è stata scartata per lo scope del progetto (un solo dominio applicativo, un singolo team di sviluppo, nessun requisito di scalabilità orizzontale differenziata per sotto-dominio), mentre un backend puramente monolitico senza separazione in strati avrebbe reso più difficile isolare e sostituire la logica di accesso ai dati (motivazione discussa per la scelta di jOOQ rispetto a un ORM).

4.5 Scelte Tecnologiche

4.6 Stack tecnologico

4.6.1 Panoramica

Livello	Tecnologia	Versione
Frontend	Java Swing (JDK puro, nessun framework)	Java 11+
Backend	Spring Boot (Web, Security, Validation)	4.1.1
Accesso ai dati	jOOQ	(spring-boot-starter-jooq)
Autenticazione	JWT (io.jsonwebtoken:jjwt)	0.11.5
Database	Postgres, ospitato su Neon	17.x

Tabella 7: Stack tecnologico del sistema

4.7 Backend

4.7.1 Linguaggio e framework

Il backend è realizzato in **Java** con **Spring Boot**:

- `spring-boot-starter-web` per l'esposizione REST
- `spring-boot-starter-security` per l'autenticazione/autorizzazione
- `spring-boot-starter-jooq` per l'accesso ai dati
- `spring-boot-starter-validation` per la validazione dei body delle richieste

L'accesso al database è effettuato tramite **jOOQ**, una libreria di query building type-safe in sintassi fluente Java, che genera le classi di accesso alle tabelle direttamente a partire dallo schema SQL reale (Sezione 4.9). L'autenticazione utilizza **JWT** (`io.jsonwebtoken:jjwt`) con token firmati HMAC e verificati da un filtro Spring Security dedicato (`JWTRequestFilter`, Sezione 4.11).

4.7.2 Motivazione

Spring Boot è stato scelto per la maturità del suo ecosistema per applicazioni REST Java (auto-configurazione, integrazione nativa con Spring Security e validazione dei dati in ingresso) e perché rispetta direttamente il vincolo tecnologico di linguaggio Object-Oriented (Sezione 3.2.2). jOOQ è stato preferito a un ORM come Hibernate/JPA perché, coerentemente con la scelta progettuale discussa in Sezione 4.9, lo schema del database è stato progettato e gestito autonomamente in SQL: jOOQ genera codice type-safe a partire da quello schema, senza richiedere che le entità Java guidino (o vincolino) la struttura delle tabelle.

4.8 Frontend

4.8.1 Framework

Il frontend di BugBoard26 è realizzato in **Java Swing**.

La comunicazione con il backend avviene esclusivamente tramite HTTP/JSON, usando il client `java.net.http.HttpClient` incluso nel JDK e il parser JSON `objectMapper`, fornito dalla libreria **Jackson**.

4.8.2 Motivazione

La scelta di Swing, anziché una Single Page Application web, deriva dalla familiarità degli sviluppatori con la struttura Java per la creazione di GUI.

4.9 Persistenza dei dati

4.9.1 Sistema di gestione dei dati

Il sistema utilizza **PostgreSQL** come DBMS relazionale, ospitato come servizio gestito su **Neon** (<https://neon.tech>), una piattaforma di Postgres serverless in cloud. Il backend si connette a Neon tramite una connection string JDBC standard, configurata tramite le variabili d'ambiente `DB_URL`, `DB_USER` e `DB_PASS` (caricate da un file `.env` locale, mai versionato, per questioni di sicurezza). Lo schema relazionale (Sezione 13) è stato creato manualmente sul database Neon tramite lo script DDL di Sezione 5.1.6; jOOQ genera poi, in fase di build, le classi Java di accesso a partire da questo schema reale (jooq-codegen-maven, connesso allo stesso database). Neon garantisce un'availability del 99.95%, corrispondente a *three nines*.

4.9.2 Motivazione

La scelta di un DBMS relazionale (anziché NoSQL) segue direttamente dal modello a entità e relazioni con vincoli di integrità referenziale forti (issue → utente creatore/assegnatario, team → progetto, collaborazione utente↔team) discusso in Sezione 4.9.1, per cui un DBMS relazionale con supporto nativo a chiavi esterne, vincoli **CHECK** ed enum a livello di schema è la scelta più diretta. All'interno delle opzioni relazionali, Neon è stato preferito a un'istanza Postgres locale o autogestita perché:

- elimina la necessità di installare e amministrare un server Postgres sulla macchina di sviluppo o su un'infrastruttura dedicata, rendendo il backend distribuibile "as is" verso qualunque ambiente che possa raggiungere Neon in rete;
- offre un piano gratuito sufficiente per lo sviluppo e la demo del progetto, senza costi di infrastruttura;
- mantiene comunque pieno supporto SQL standard e compatibilità con jOOQ, driver JDBC PostgreSQL incluso, senza richiedere alcuna modifica al codice di accesso ai dati rispetto a un Postgres autogestito.

In maniera simile alle scelte per il frontend, anche questa scelta è stata incentivata dalla familiarità con i DBMS relazionali classici.

4.10 Comunicazione e protocolli

4.10.1 Protocollo di comunicazione

La comunicazione tra frontend e backend avviene esclusivamente tramite **REST su HTTP/1.1**, interamente stateless: ogni interazione utente che richiede dati dal server (login, lista issue, dashboard, ecc.) corrisponde a una singola richiesta HTTP sincrona.

4.10.2 Formato dei dati

Il formato di scambio è **JSON** sia in richiesta che in risposta. I corpi delle richieste vengono prodotti/consumati tramite Jackson (incluso in `spring-boot-starter-web`); per la gestione delle immagini si sfrutta invece l'header `Content-Type: image/jpeg`.

4.10.3 Motivazione

REST/JSON è stato scelto perché è lo standard de-facto per API consumate da client eterogenei (nel nostro caso un client desktop Swing, ma potenzialmente sostituibile da qualunque altro client HTTP per RNF-010), è nativamente supportato da Spring Web senza configurazione aggiuntiva, ed è sufficiente per il pattern di interazione richiesta/risposta di tutti i casi d'uso di Sezione 3.6, che non richiedono notifiche push né comunicazione bidirezionale in tempo reale.

4.11 Sicurezza

4.11.1 Autenticazione e autorizzazione

L'autenticazione è basata su **JWT** (JSON Web Token): al login (`POST /auth/login`), verificate email e password (confronto dell'hash BCrypt tramite `Spring Security PasswordEncoder`), il backend genera un token firmato HMAC (libreria `io.jsonwebtoken:jjwt`) contenente l'identità dell'utente (`subject`) e il suo ruolo come claim, con scadenza di 8 ore. Il frontend allega il token a ogni richiesta successiva nell'header `Authorization: Bearer <token>`; lato backend, un filtro dedicato (`JWTRequestFilter`, eseguito prima dei controller) verifica firma e scadenza del token e popola il contesto di sicurezza di Spring. L'autorizzazione è basata sui ruoli (`DEV/ADMIN`, coerenti con l'enum `user_type` dello schema): gli endpoint riservati all'amministratore sono annotati con `@PreAuthorize("hasRole('ADMIN')`", mentre tutti gli altri endpoint richiedono semplicemente un token valido (configurazione `Security`, che consente l'accesso non autenticato solo alle rotte `/auth/**`).

4.11.2 Protezione dei dati

Le password non sono mai memorizzate né trasmesse in chiaro: subiscono l'effetto di una funzione di hash tramite **BCrypt** prima della persistenza (RNF-007) e, quando un profilo utente viene restituito dalle API, il campo della password hashata è esplicitamente oscurato dal backend prima della serializzazione JSON. Gli errori applicativi sono gestiti da un `@RestControllerAdvice` centralizzato (Sezione 5.3.11) che restituisce messaggi di errore generici verso il client, senza esporre dettagli interni dell'eccezione originale (es. stack trace o messaggi SQL grezzi).

4.12 Interfacce API

Il backend espone le API REST elencate in Tabella 8, tutte protette da JWT tranne `/auth/login`.

Metodo	Endpoint	Auth	Descrizione
POST	<code>/auth/login</code>	Nessuna	Autenticazione utente.
POST	<code>/users</code>	JWT + Admin	Creazione nuovo utente.
POST	<code>/issues</code>	JWT	Creazione nuova issue.
GET	<code>/issues</code>	JWT	Lista delle issue.
GET	<code>/issues/suggestion</code>	JWT	Suggerimento carico di lavoro.
GET	<code>/issues?assignee=me/all&type=&status=&priority=&sort=</code>	JWT	Filtraggio issue.
PUT	<code>/issues/{id}/handle</code>	JWT	Gestione di una issue.
PUT	<code>/issues/{id}/resolve</code>	JWT	Risoluzione di una issue.
GET	<code>/issues/{id}/image</code>	JWT	Recupero immagine della issue.
POST	<code>/projects</code>	JWT + Admin	Creazione nuovo progetto con team.
GET	<code>/projects</code>	JWT	Lista dei progetti.
GET	<code>/projects?mine=true</code>	JWT	Lista dei propri progetti.
GET	<code>/projects/{id}/teams</code>	JWT	Team associati al progetto.
PUT	<code>/projects/{id}/teams/{id}</code>	JWT	Entrata in un team.
GET	<code>/dashboard</code>	JWT + Admin	Dati della dashboard.
GET	<code>/reports/month=&year=&project=</code>	JWT + Admin	Generazione report mensile.

Tabella 8: Endpoint REST esposti dal backend

4.13 Gestione degli Aspetti Non Funzionali

4.13.1 Performance

Le operazioni comuni (RNF-001/002) sono servite da query jOOQ dirette e mirate: ad esempio `GET /issues` esegue un'unica query con `JOIN` verso `Collaboration/Team` per limitare il risultato ai soli progetti dell'utente, invece di recuperare tutte le issue e filtrarle in memoria lato applicativo.

4.13.2 Scalabilità

Il backend è **stateless**: l'identità dell'utente autenticato è interamente contenuta nel token JWT verificato a ogni richiesta (Sezione 4.11), non in una sessione server-side. La scalabilità del database è demandata a Neon, che offre scaling automatico delle risorse di calcolo lato Postgres senza intervento manuale.

4.13.3 Manutenibilità

La manutenibilità è favorita principalmente da tre scelte di design discusse altrove: la separazione in package per responsabilità sia lato backend sia lato frontend, l'uso sistematico di DTO dedicati che disaccoppiano il modello di persistenza dal contratto REST, e l'applicazione dei principi SOLID (Sezione 5.3.3).

5 Design del Software

5.1 Schema di Persistenza dei Dati

5.1.1 Introduzione

Le scelte progettuali per la persistenza dei dati sono discusse nella seguente sezione. La scelta è ricaduta su un classico DBMS relazionale progettato e gestito autonomamente, per avere completo controllo sulla struttura delle tabelle, a differenza di strumenti come JPA che definiscono uno schema sulla base delle classi utilizzate nel modulo Service. È stata utilizzata la libreria jOOQ per il querying dinamico in sintassi fluente, che consente di standardizzare la scrittura del codice che interagisce con il modulo Repository.

5.1.2 Progettazione Concettuale

Sulla base dei requisiti precedentemente elencati, di seguito le tre principali entità di dominio e le relazioni che intercorrono tra esse:

Entità e Relazioni

Gli oggetti del dominio sono stati modellati attraverso l'individuazione di **Entità** e **Relazioni**, con i rispettivi attributi.

Entità

User

Rappresenta un utente generico della piattaforma:

- [Email](#)
- HashedPassword

Admin

Rappresenta un utente con maggiori privilegi tra cui la creazione di progetti e di teams

Issue

Identifica una singola issue registrata nel sistema:

- [Title](#)
- [Description](#)
- Priority
- Status
- Type
- Image
- Lista di Tag
- CreatedAt
- DoneAt

Team

Rappresenta un team di sviluppatori che si occupa di segnalare e gestire le issue:

- [Name](#)

Project

Rappresenta un progetto a cui un ipotetico team intende lavorare per la risoluzione di eventuali bug

- [Name](#)

Relazioni

- **Creation**
- **Handling**

Convenzioni Notazionali

Nome → Entità o Relazione

Nome → Attributo dell'Entità o della Relazione

[Attributo](#) → Attributo Identificativo dell'Entità

Gerarchie tra Entità

Durante la progettazione concettuale è stata individuata l'entità generale **User**, dalla quale deriva un'entità specializzazione: **Admin**. Tale gerarchia consente di modellare in maniera più accurata i diversi ruoli che gli utenti possono assumere all'interno della piattaforma **Bugboard26**.

Tale gerarchia è caratterizzata da un **vincolo di completezza parziale**.

5.1.3 Diagramma Concettuale

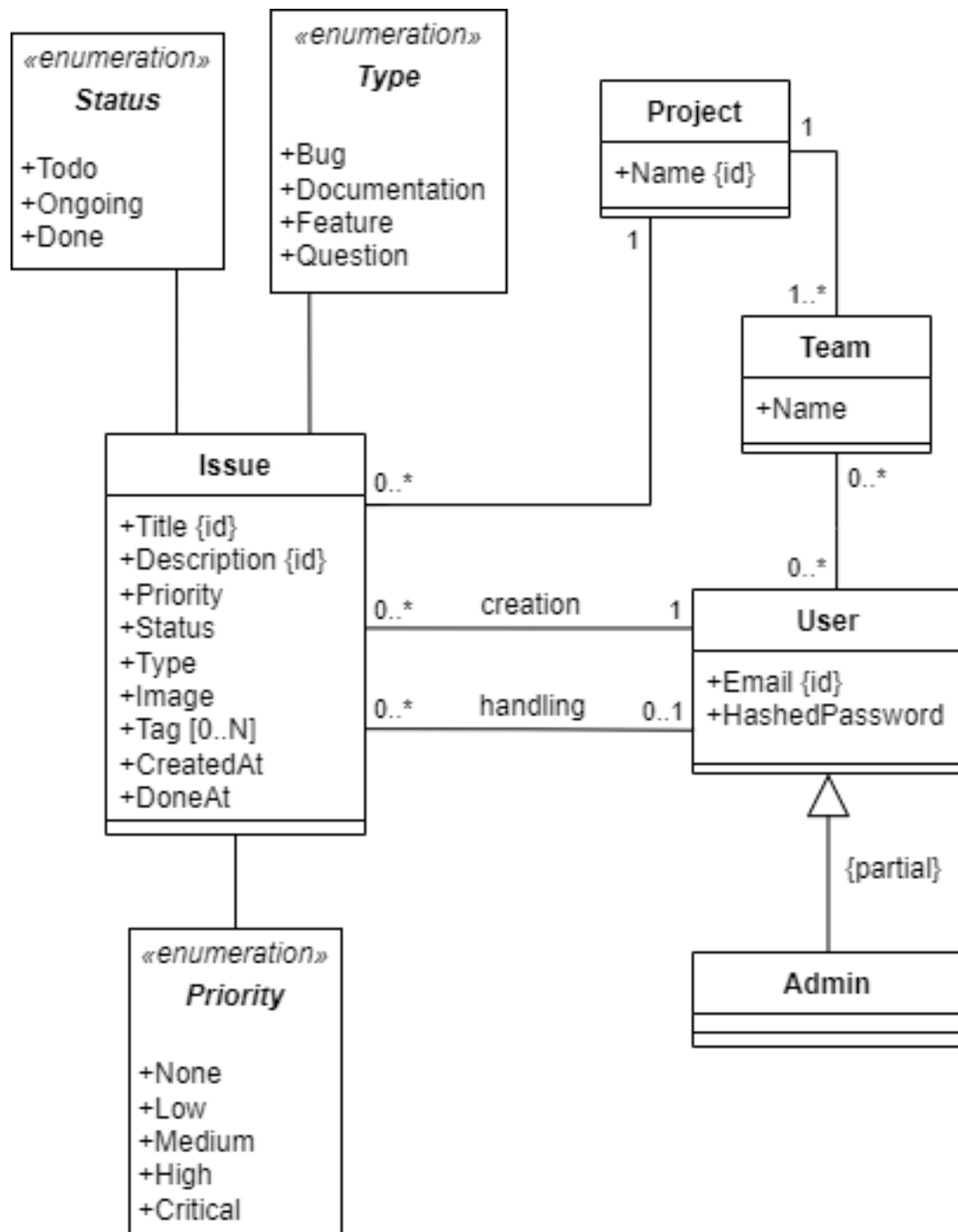


Figura 12: Diagramma UML concettuale per la persistenza dei dati

5.1.4 Ristrutturazione

La determinazione del modello concettuale precedentemente illustrata deve essere seguita da un processo di traduzione tale da portare a stabilire uno **schema logico**. Quest'ultimo processo è definito *ristrutturazione del modello concettuale*, una fase il cui obiettivo è quello di definire uno schema logico in grado di descrivere tutte le informazioni contenute nello schema concettuale prodotto dalla fase di progettazione concettuale.

Le fasi che hanno costituito la ristrutturazione sono state le seguenti:

- **Analisi delle ridondanze:** non sono state individuate ridondanze da eliminare.
- **Eliminazione delle generalizzazioni:** a tal proposito è stato utilizzato un tipo di accorpamento "entità figlie nell'entità padre", quindi l'entità *Admin* è stata assorbita da *User* poiché la gerarchia presenta un **vincolo di completezza parziale**. È stato di conseguenza aggiunto l'attributo `user_type` all'entità **User** per differenziare le due tipologie di utenze.
- **Partizionamento/accorpamento di entità:** ritenuto non necessario.
- **Eliminazione degli attributi multivalore:** la *Lista di Tags* è stata trasformata in una stringa unica, alla quale verrà apportato un parsing da parte del modulo Service.
- **Scelta degli identificatori principali:** sono stati definiti identificatori automaticamente serializzati per ogni entità, per facilitare i vincoli di integrità referenziale e la loro propagazione.

Alla pagina seguente è rappresentato il diagramma UML ristrutturato secondo i suddetti dettagli:

Diagramma Concettuale UML Ristrutturato

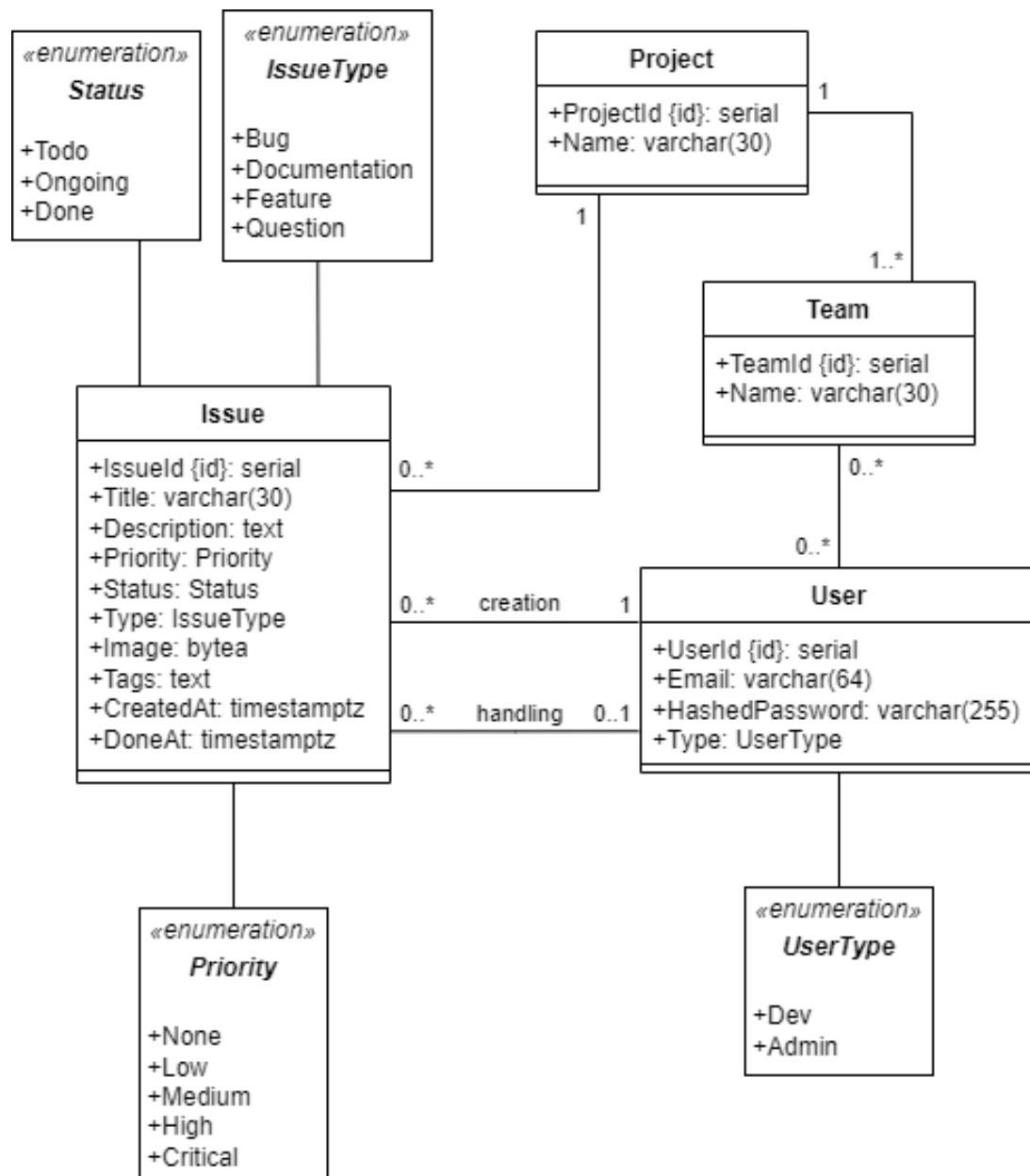


Figura 13: Diagramma UML ristrutturato per la persistenza dei dati

5.1.5 Schema Logico-Relazionale

Il seguente schema logico-relazionale rappresenta il **risultato** del processo di **mapping logico-relazionale** applicato agli elementi costituenti lo schema concettuale ristrutturato precedentemente illustrato.

La creazione della tabella intermedia **Collaboration** rappresenta la relazione di cardinalità N:N presente tra **User** e **Team**. La tabella **Issue** viene invece arricchita con **creator_id** e **assignee_id**, per rappresentare la relazione di creazione e gestione issue mentre **project_id** per indicare il progetto di riferimento del problema segnalato. Similmente, anche **Team** riceve un **project_id** per rappresentare la sua appartenenza in un progetto.

```
User_          (user_id, email, hashed_password, type)
Project        (project_id, name)
Issue          (issue_id, title, description, priority, status, type, image, tags,
               created_at, done_at, creator_id, assignee_id, project_id)
  Issue.creator_id → User_.user_id
  Issue.assignee_id → User_.user_id
  Issue.project_id → Project.project_id
Team           (team_id, name, project_id)
  Team.project_id → Project.project_id
Collaboration  (team_id, user_id)
  Collaboration.team_id → Team.team_id
  Collaboration.user_id → User_.user_id
```

Convenzioni Notazionali

Attributo → Chiave Primaria

Attributo → Chiave Esterna

5.1.6 Script DDL

Lo script DDL utilizzato per la creazione del database è situato nella repository GitHub, il cui link è situato nella sezione Repository GitHub

5.2 Diagramma delle Classi di Design

Il diagramma delle classi descrive la struttura statica di BugBoard26: mostra le classi principali, le rispettive responsabilità e le relazioni che ne organizzano la collaborazione. Per mantenere chiara la separazione architetturale del sistema, la descrizione distingue il modulo **frontend** dal modulo **backend**. I due moduli collaborano tramite API REST e scambio di dati in formato JSON, senza dipendenze dirette tra le rispettive classi.

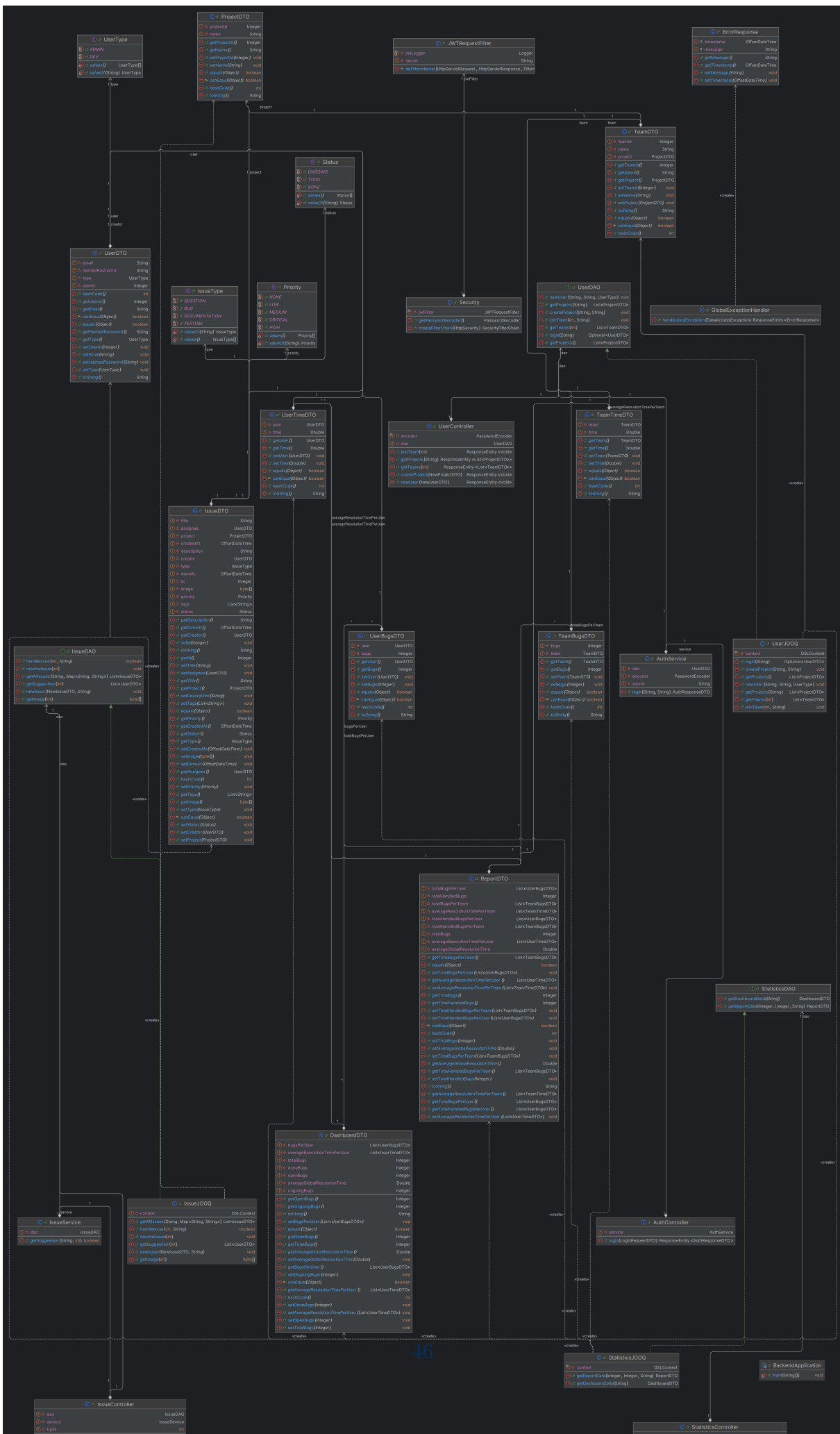
Frontend Il diagramma del frontend rappresenta le classi che compongono l'interfaccia desktop e le loro collaborazioni. **LoginScreen** gestisce l'accesso prima dell'autenticazione; le schermate operative (**Hub**, **IssuesListScreen**, **NewIssueScreen**, **AdminDashboardScreen**, **ReportsScreen**, **CreateProjectScreen**, **CreateUserScreen** e **ChooseProjectScreen**) ereditano da **BaseFrame**, che fornisce la struttura comune e la navigazione. Le schermate delegano le operazioni ai controller del frontend, i quali preparano le richieste e si appoggiano a **ApiClient** per la comunicazione HTTP. **Session** mantiene lo stato di autenticazione, mentre i DTO rappresentano i dati scambiati con il backend. Il diagramma evidenzia inoltre attributi, metodi e relazioni tra queste classi.

Backend Il diagramma del backend mostra le classi che ricevono e gestiscono le richieste REST. **AuthController**, **IssueController**, **UserController** e **StatisticsController** espongono le operazioni del servizio e collaborano con i componenti applicativi e di accesso ai dati. **AuthService** e **IssueService** racchiudono le operazioni di servizio; le interfacce DAO definiscono i contratti per l'accesso ai dati e le classi **postgresjooqimpl** li realizzano tramite jOOQ. Le classi DTO descrivono i dati restituiti o ricevuti dagli endpoint. Le classi di sicurezza e di gestione delle eccezioni completano il modello, mostrando le responsabilità trasversali del backend. Anche in questa vista sono riportati metodi, attributi e relazioni tra le classi.

5.2.1 Panoramica dei package

Il codice è diviso in due moduli Maven indipendenti (**backend** e **frontend**, con **pom.xml** separati), ciascuno organizzato internamente per responsabilità: il backend nei package **controller**, **service**, **dao**, **postgresjooqimpl**, **dto**, **security**, **exception**; il frontend nei package **gui**, **controller**, **dto**, **config**, **exception**). Le classi **dto** dei due moduli sono definite separatamente e non condivise: pur rappresentando spesso concetti analoghi (es. **IssueDTO** esiste in entrambi), ciascun modulo ha la propria copia, così da non introdurre una dipendenza di compilazione diretta tra backend e frontend che violerebbe RNF-010.

Le seguenti immagini sono disponibili nella repository GitHub per una migliore fruibilità.



5.3 Scelte di Software Design

5.3.1 Pattern architetturale

A livello di design del software, sia il backend sia il frontend seguono una variante semplificata di **layered architecture**: nel backend, i tre strati sono `@RestController` (presentazione HTTP), `Service` (astrazione del dominio) e `postgresjooqimpl` che implementano le interfacce `DAO` (accesso ai dati); nel frontend, gli strati sono le schermate del package `gui` (presentazione), i controller di dominio del package `controller` (logica applicativa di comunicazione) e `ApiClient` (trasporto HTTP/JSON).

5.3.2 Design Pattern utilizzati

5.3.3 Repository Pattern

- **Contesto:** Accesso al layer di persistenza, lato backend
- **Problema:** Accoppiamento tra i controller REST e il dettaglio delle query SQL/jOOQ
- **Soluzione:** Introduzione di interfacce che astraggono le operazioni di dominio, con un'unica implementazione concreta basata su jOOQ

5.3.4 Dependency Injection

- **Contesto:** Gestione delle dipendenze tra componenti, lato backend
- **Problema:** Accoppiamento rigido tra classi che ne riduce la testabilità
- **Soluzione:** Iniezione delle dipendenze tramite costruttore, gestita dal container Spring

5.3.5 DTO Pattern

- **Contesto:** Comunicazione tra backend e frontend via API REST
- **Problema:** Esposizione diretta del dominio verso il livello di presentazione
- **Soluzione:** Introduzione di oggetti DTO dedicati per il trasporto dei dati

5.3.6 Singleton

- **Contesto:** Comunicazione del frontend con le API REST del backend e gestione dello stato di autenticazione
- **Problema:** Senza un punto di accesso unico, ogni schermata dovrebbe costruire da sé richieste HTTP, gestire token ed errori di rete, duplicando logica e accoppiando la UI ai dettagli di trasporto
- **Soluzione:** `ApiClient` agisce da *facade* verso l'intera API REST (costruzione delle richieste, header di autenticazione, traduzione degli errori HTTP in `ApiException`); `Session` è un *singleton* statico che mantiene lo stato di autenticazione corrente, letto da ogni schermata senza doverlo propagare esplicitamente

5.3.7 Esempi di Principi SOLID

5.3.8 Single Responsibility Principle (SRP)

Nel frontend, ogni classe ha un'unica ragione per cambiare:

- le classi del package `gui` si occupano solo di costruzione dell'interfaccia e gestione degli eventi utente, mai di dettagli HTTP;
- `ApiClient` si occupa solo del trasporto HTTP/JSON, non conosce alcuna regola di dominio (es. non decide se un utente può creare un progetto: si limita a inoltrare la richiesta e a restituire l'esito);
- i controller di dominio (`IssueController`, `UserController`, ...) traducono le operazioni di un singolo dominio in chiamate a `ApiClient`, senza mai costruire direttamente una `HttpRequest`;
- `Session` si occupa solo dello stato di autenticazione, non della navigazione tra schermate (di cui è responsabile `BaseFrame`).

Nel backend vale lo stesso principio: i `@RestController` si occupano solo di mapping HTTP e validazione dell'input, mai di query SQL; le implementazioni `postgresjooqimpl` si occupano solo della traduzione delle operazioni DAO in query jOOQ, senza conoscere regole di autorizzazione; `JWTRequestFilter` si occupa solo della verifica del token, non decide quali ruoli possano accedere a un dato endpoint (decisione demandata alle annotazioni `@PreAuthorize` sui controller).

5.3.9 Open/Closed Principle (OCP)

`BaseFrame` è aperta all'estensione (ogni nuova schermata la estende per ereditare dimensione di default, stato massimizzato e `navigateTo(...)`) ma chiusa alla modifica: aggiungere una nuova schermata (es. una futura `NotificationsScreen`) non richiede alcuna modifica a `BaseFrame` né alle schermate esistenti. Analogamente, aggiungere un nuovo controller di dominio che usa `ApiClient.call(...)` non richiede modifiche ad `ApiClient`. Lato backend, aggiungere un nuovo endpoint (es. `DELETE /issues/{id}`) richiede solo un nuovo metodo in `IssueController` e in `IssueDAO/IssueJOOQ`: non è necessaria alcuna modifica a `Security` o `JWTRequestFilter`, che restano chiusi rispetto all'aggiunta di nuove rotte autenticate.

5.3.10 Liskov Substitution Principle (LSP)

Tutte le sottoclassi di `BaseFrame` (`Hub`, `IssuesListScreen`, `NewIssueScreen`, ecc.) sono utilizzabili ovunque sia atteso un `JFrame` standard (ad esempio come argomento di `navigateTo(JFrame next)`), senza che il chiamante debba conoscere la sottoclasse concreta: il codice di navigazione tratta qualunque schermata autenticata in modo uniforme. Lato backend, `UserJOOQ` e `IssueJOOQ` sono usate dai controller esclusivamente attraverso i tipi `UserDAO/IssueDAO`: qualunque implementazione alternativa di queste interfacce (es. una implementazione in-memory per i test) potrebbe sostituirle senza alcuna modifica ai controller, che dipendono solo dal contratto dichiarato dall'interfaccia.

5.3.11 Gestione degli errori e delle eccezioni

Il backend centralizza la gestione degli errori in un unico

`@RestControllerAdvice (GlobalExceptionHandler)`

che intercetta le eccezioni sollevate durante l'accesso ai dati (in particolare le `DataAccessException` di Spring, che avvolgono sia errori jOOQ/JDBC sia le eccezioni applicative sollevate esplicitamente dalla stored procedure `add_new_teams`, Sezione 5.1.6) e le traduce in una risposta JSON uniforme `{message, timestamp}`. Questo evita che i controller debbano gestire try/catch espliciti e, allo stesso tempo, impedisce che dettagli interni (query SQL, stack trace) raggiungano il client.

Le eccezioni non catturate esplicitamente da questo handler (es. un token JWT assente o non valido) ricadono invece nella gestione di default di Spring Security/Spring Boot, che restituisce comunque un corpo JSON (con chiave `error` anziché `message`): per questo motivo il frontend verifica entrambe le chiavi quando interpreta un errore.

5.3.12 Strategie di sicurezza applicativa

Oltre all'autenticazione JWT e all'hashing BCrypt delle password già descritti in Sezione 4.11, il backend valida gli input in ingresso tramite le annotazioni `@Valid` di Spring Validation sui body delle richieste (es. formato dell'email e presenza di email/password nel login), rifiutando con `400 Bad Request` le richieste malformate prima che raggiungano il livello di accesso ai dati. L'uso esclusivo di query parametriche jOOQ (nessuna concatenazione di stringhe SQL) elimina strutturalmente il rischio di SQL injection.

5.4 Versioning e Repository

La repository GitHub è raggiungibile dal seguente link:

<https://github.com/TrialShock26/BugBoard26>

5.4.1 Strategia di branching

Lo sviluppo è avvenuto su branch dedicati secondo il *feature branch workflow*, integrati in `main` tramite pull request e successivamente eliminati, come testimoniato dai commit di merge nella cronologia del repository.

5.4.2 Frequenza dei commit nel tempo

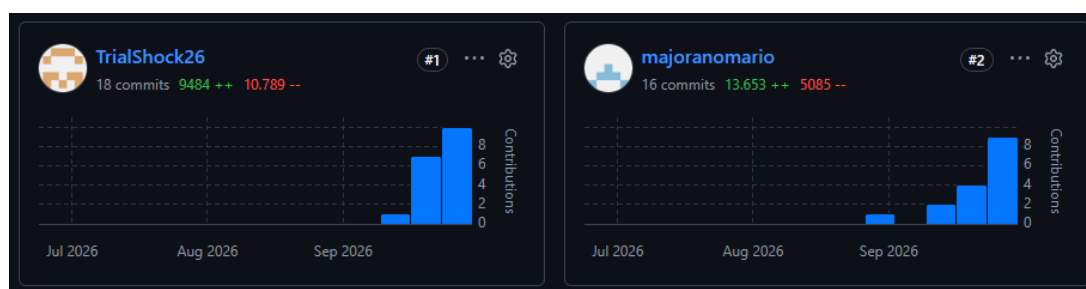


Figura 16: Grafico della frequenza dei commit nel tempo

5.5 Qualità del Codice

5.5.1 Strumento di analisi: SonarQube

L'analisi statica di SonarQube ha individuato solo 7 errori:

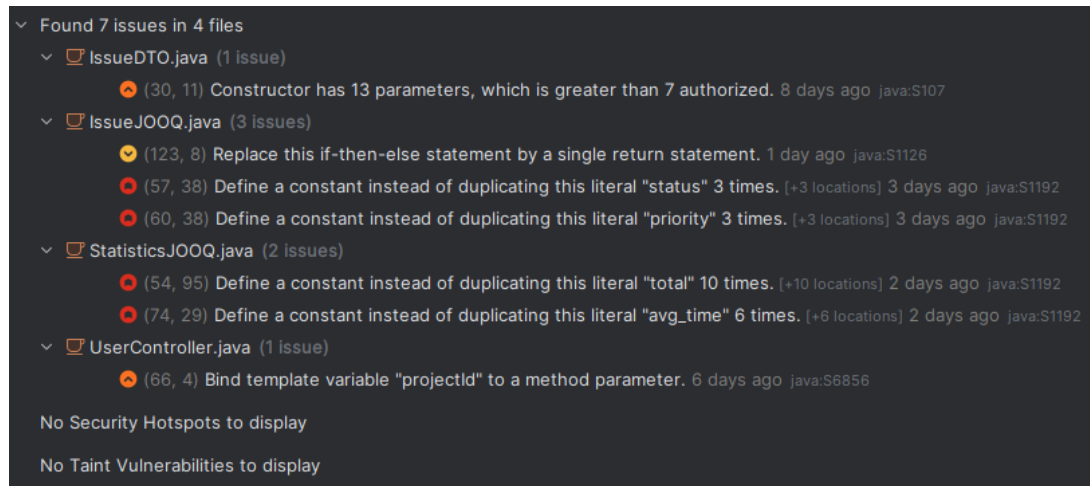


Figura 17: Report SonarQube

Il report restituisce alcune segnalazioni circoscritte nei file analizzati. Si tratta prevalentemente di suggerimenti per rendere il codice più uniforme e agevole da mantenere, come riuso di stringhe costanti e semplificazione di alcune dichiarazioni.

Le segnalazioni non indicano, di per sé, malfunzionamenti osservati: sono indicazioni automatiche da valutare nel contesto del progetto. Il report offre quindi spunti di rifinitura del codice e una base per eventuali interventi di manutenzione successivi.

5.6 Build Automation

5.6.1 File di build — pom.xml

Il software è orchestrato da Maven, per gestire in modo centralizzato tutte le dipendenze necessarie alla compilazione.

Sia il backend che il frontend sfruttano questo strumento di build automation.

6 Attività di Testing

L'attività di testing ha lo scopo di verificare che BugBoard26 soddisfi i requisiti descritti in questo documento e che le sue funzionalità producano i risultati attesi nelle condizioni d'uso previste. La verifica considera sia i percorsi che si concludono correttamente sia le situazioni in cui dati mancanti o condizioni sfavorevoli impediscono il completamento di un'operazione. In questa sezione viene presentato un test plan dedicato alla creazione di una nuova issue (`NewIssue`). I casi di prova sono definiti a partire dai requisiti e dal caso d'uso corrispondente: descrivono le precondizioni, le azioni dell'utente e gli esiti attesi. Il piano costituisce quindi un riferimento per valutare la funzionalità in modo sistematico e tracciabile.

6.1 Test plan

6.1.1 Funzionalità scelta e motivazione

Il test plan riguarda la funzionalità di creazione di una nuova issue (`NewIssue`), corrispondente al caso d'uso UC3, "Segnala un issue", e collegata ai requisiti RF-004, RF-005 e RF-006. La scelta è motivata dal ruolo centrale che la segnalazione riveste in BugBoard26: costituisce il punto di partenza per la gestione successiva delle issue. Inoltre, il flusso consente di verificare, all'interno di un'unica funzionalità, l'inserimento dei dati essenziali, la classificazione della issue, l'eventuale specificazione della priorità e l'aggiunta di un'immagine. Il suo perimetro è abbastanza delimitato da rendere i casi di prova comprensibili e ripetibili, pur coprendo sia il percorso di successo sia le principali condizioni di errore. La creazione è stata preferita alla modifica dello stato perché permette di verificare più requisiti funzionali direttamente collegati alla segnalazione iniziale: la presenza di titolo e descrizione (RF-004), l'assegnazione di una tipologia (RF-006) e la possibilità di indicare priorità e immagine (RF-005). La modifica dello stato, pur rilevante, riguarda una fase successiva del ciclo di vita della issue e non coprirebbe questi requisiti di creazione.

6.1.2 Obiettivo e perimetro

L'obiettivo del piano è stabilire se un utente autenticato riesce a creare una issue associata al progetto scelto, con i dati indicati, e se il sistema comunica in modo comprensibile l'esito dell'operazione. In caso di creazione riuscita, la issue deve essere registrata e consultabile nell'elenco, con stato iniziale `TODO`, come specificato dal criterio di accettazione di RF-004.

Il piano considera i seguenti aspetti:

- inserimento dei dati richiesti e trattamento dei campi mancanti;
- selezione di una tipologia e di un progetto;
- inserimento facoltativo della priorità, dei tag e di un'immagine;
- conferma della creazione e consultazione della issue risultante;
- comportamento atteso in caso di indisponibilità del servizio o di interruzione durante la creazione.

Il piano si limita al flusso di creazione e alla verifica del risultato visibile all'utente. Non comprende la modifica o l'assegnazione della issue, la gestione del suo stato dopo la creazione, né la valutazione delle prestazioni e dell'usabilità generale del prodotto.

6.1.3 Precondizioni e dati di prova

Prima dell'esecuzione occorre disporre di un account autenticato e di almeno un progetto selezionabile. L'elenco delle issue deve essere consultabile così da poter controllare la presenza della nuova segnalazione. Per rendere distinguibili i risultati, ogni caso che prevede la creazione usa un titolo univoco.

I dati di prova comprendono:

- un titolo e una descrizione non vuoti;
- una tipologia valida tra `BUG`, `QUESTION`, `DOCUMENTATION` e `FEATURE`;
- un progetto disponibile;
- valori facoltativi di priorità, tag e immagine, da usare nei casi che li prevedono.

6.1.4 Casi di prova

I casi seguenti descrivono le condizioni iniziali, l'azione dell'utente e il risultato atteso. Si assume che, quando non diversamente specificato, l'utente sia autenticato e abbia selezionato un progetto disponibile.

ID	Scenario	Azione	Esito atteso
TC01	Creazione con i soli dati essenziali	Inserire titolo e descrizione, selezionare tipologia e progetto, lasciare vuoti i campi facoltativi e confermare.	La issue viene creata una sola volta, compare nell'elenco con i dati inseriti e presenta stato TODO.
TC02	Titolo mancante	Compilare gli altri dati richiesti lasciando vuoto il titolo e confermare.	La creazione viene impedita; l'utente riceve un'indicazione del dato mancante e nessuna issue viene aggiunta all'elenco.
TC03	Descrizione mancante	Compilare gli altri dati richiesti lasciando vuota la descrizione e confermare.	La creazione viene impedita; l'utente riceve un'indicazione del dato mancante e nessuna issue viene aggiunta all'elenco.
TC04	Tipologia non selezionata	Inserire titolo e descrizione senza selezionare una tipologia e confermare.	La creazione non viene completata; il sistema segnala che occorre indicare una tipologia e non compare una nuova issue nell'elenco.
TC05	Progetto non selezionato	Inserire titolo e descrizione e selezionare una tipologia, senza indicare un progetto, quindi confermare.	La creazione non viene completata; il sistema segnala che occorre indicare un progetto e non compare una nuova issue nell'elenco.
TC06	Creazione con priorità	Compilare i dati richiesti, selezionare una priorità e confermare.	La issue viene creata con la priorità selezionata, oltre che con i restanti dati inseriti.
TC07	Creazione con tag	Compilare i dati richiesti e inserire uno o più tag, quindi confermare.	La issue viene creata e i tag indicati risultano associati alla segnalazione.
TC08	Creazione con immagine	Compilare i dati richiesti, selezionare un'immagine e confermare.	La issue viene creata e l'immagine risulta associata alla segnalazione.
TC09	Creazione con tutti i dati facoltativi	Compilare i dati richiesti e specificare priorità, tag e immagine, quindi confermare.	La issue viene creata con tutti i dati indicati, senza perdita o sostituzione dei valori inseriti.
TC10	Indisponibilità durante la creazione	Dopo aver compilato i dati, interrompere la disponibilità del servizio necessario al completamento e confermare.	L'utente riceve un messaggio di errore comprensibile; non viene mostrato un esito positivo e la issue non compare nell'elenco.

ID	Scenario	Azione	Esito atteso
TC11	Nuovo tentativo dopo un errore	Dopo un tentativo non riuscito, ripristinare la disponibilità del servizio e ripetere la conferma con dati validi.	Il nuovo tentativo produce una sola issue nell'elenco e mostra l'esito positivo dell'operazione.

6.1.5 Strategie di progettazione dei test

La progettazione dei casi di prova è stata svolta seguendo criteri sistematici, con l'obiettivo di individuare quali dati e situazioni verificare e di motivare quando la copertura del flusso possa considerarsi sufficiente. I casi sono stati ricavati dai requisiti RF-004, RF-005 e RF-006 e dal caso d'uso UC3, senza fare riferimento all'organizzazione interna del sistema. La prima strategia consiste nella suddivisione degli input in classi di equivalenza, secondo un approccio *black-box*. Il dominio dei dati viene ripartito in gruppi per i quali ci si attende un comportamento uniforme. Si seleziona quindi un valore rappresentativo per ciascun gruppo, concentrando le verifiche sui comportamenti distinti, invece di ripetere prove equivalenti. La seconda strategia consiste nella copertura dei percorsi funzionali descritti dai requisiti e dal caso d'uso. Per ciascuna condizione rilevante del flusso si considera il percorso in cui essa è soddisfatta e, quando applicabile, quello in cui non lo è. In questo modo vengono verificati sia il completamento dell'operazione sia i principali percorsi alternativi, come l'input incompleto o l'indisponibilità del servizio. Entrambe le strategie sono state applicate alla creazione di una issue. La suddivisione in classi è adatta a distinguere i dati richiesti da quelli facoltativi e i valori presenti da quelli mancanti. La copertura dei percorsi funzionali permette invece di verificare che l'operazione abbia esito positivo solo quando le condizioni previste sono soddisfatte, e che un errore non venga presentato come una creazione riuscita.

Classi di equivalenza dei dati di una issue La creazione richiede un titolo, una descrizione, una tipologia e un progetto. Priorità, tag e immagine sono invece informazioni facoltative. Le classi individuate e i relativi comportamenti attesi sono riportati nella tabella seguente.

Classe	Rappresentante	Comportamento atteso
Titolo e descrizione presenti	“Errore accesso”; descrizione non vuota	La creazione può proseguire, se sono presenti anche tipologia e progetto (TC01).
Titolo mancante	Titolo vuoto	La creazione è impedita e il sistema segnala il dato richiesto (TC02).
Descrizione mancante	Descrizione vuota	La creazione è impedita e il sistema segnala il dato richiesto (TC03).
Tipologia selezionata	BUG	La issue viene creata con la tipologia indicata (TC01).
Tipologia non selezionata	Nessuna tipologia	La creazione è impedita e il sistema richiede la selezione (TC04).
Progetto selezionato	Un progetto disponibile	La issue viene associata al progetto indicato (TC01).
Progetto non selezionato	Nessun progetto	La creazione è impedita e il sistema richiede la selezione (TC05).
Priorità indicata	HIGH	La issue viene creata con la priorità selezionata (TC06).
Priorità omessa	Nessuna scelta esplicita	La creazione è consentita senza una priorità indicata (TC01).
Tag indicati	login, accesso	I tag risultano associati alla issue creata (TC07).
Tag omessi	Campo tag vuoto	La creazione è consentita senza tag (TC01).
Immagine selezionata	Un'immagine valida	L'immagine risulta associata alla issue creata (TC08).
Immagine omessa	Nessun allegato	La creazione è consentita senza immagine (TC01).
Tutti i campi facoltativi presenti	Priorità, tag e immagine indicati	La issue viene creata conservando tutte le informazioni fornite (TC09).

I casi TC02 e TC03 trattano separatamente titolo e descrizione perché sono entrambi richiesti: verificandoli distintamente è possibile accertare che l'assenza di ciascuno venga rilevata. TC04 e TC05 applicano lo stesso criterio a tipologia e progetto, anch'essi necessari per completare il flusso di creazione. Per i dati facoltativi è stato previsto sia un caso senza informazioni aggiuntive (TC01), sia una verifica separata per ciascun dato (TC06, TC07 e TC08). TC09 controlla infine la loro presenza simultanea. Questa organizzazione permette di distinguere un eventuale problema relativo a un singolo campo da uno che si manifesta quando più informazioni facoltative vengono fornite insieme. Poiché i requisiti non stabiliscono una lunghezza minima o massima per titolo e descrizione, non vengono introdotti limiti numerici arbitrari. Si considera tuttavia utile includere come casi limite i valori composti soltanto da spazi: pur non essendo stringhe vuote, non contengono informazioni significative per una segnalazione. Si propongono pertanto i seguenti casi aggiuntivi:

- **TC12 – Titolo composto soltanto da spazi:** inserire una descrizione valida e un titolo senza caratteri diversi dagli spazi. La creazione deve essere impedita e il titolo deve essere segnalato come mancante.
- **TC13 – Descrizione composta soltanto da spazi:** inserire un titolo valido e una descrizione senza caratteri diversi dagli spazi. La creazione deve essere impedita e la descrizione deve essere segnalata come mancante.

Percorsi funzionali della creazione Il flusso di creazione può essere descritto attraverso percorsi funzionali distinti. La tabella seguente riassume le condizioni considerate e il risultato atteso per ciascun percorso.

Percorso	Condizione	Esito atteso
P1	Tutti i dati richiesti sono presenti; i campi facoltativi sono omessi	Creazione della issue con i dati essenziali e stato iniziale TODO (TC01).
P2	Il titolo è assente	Creazione impedita; nessuna nuova issue compare nell'elenco (TC02).
P3	La descrizione è assente	Creazione impedita; nessuna nuova issue compare nell'elenco (TC03).
P4	La tipologia non è stata selezionata	Creazione impedita; il sistema richiede la tipologia (TC04).
P5	Il progetto non è stato selezionato	Creazione impedita; il sistema richiede il progetto (TC05).
P6	I dati richiesti sono presenti e viene indicata una priorità	Creazione della issue con la priorità scelta (TC06).
P7	I dati richiesti sono presenti e vengono indicati tag	Creazione della issue con i tag inseriti (TC07).
P8	I dati richiesti sono presenti e viene selezionata un'immagine	Creazione della issue con l'immagine associata (TC08).
P9	I dati richiesti e tutti i dati facoltativi sono presenti	Creazione della issue con tutte le informazioni inserite (TC09).
P10	Il servizio non è disponibile durante la conferma	La creazione non viene presentata come riuscita e l'utente riceve un messaggio di errore (TC10).
P11	Dopo un errore, il servizio torna disponibile e l'utente riprova	Il nuovo tentativo crea una sola issue e comunica l'esito positivo (TC11).

I percorsi P1–P9 coprono l'inserimento dei dati essenziali e le combinazioni rilevanti dei campi facoltativi. P10 e P11 verificano invece che il fallimento dell'operazione sia riconoscibile e che un tentativo successivo possa concludersi correttamente. La verifica dell'elenco è importante in entrambi i casi: nei percorsi riusciti deve essere presente la nuova issue, mentre nei percorsi interrotti o rifiutati non deve risultare una creazione non confermata. Questo criterio di copertura è riferito ai comportamenti descritti dalla specifica e non ai percorsi interni del programma. In questo modo il test plan resta indipendente dalle scelte implementative e può essere usato per verificare la funzionalità attraverso gli esiti visibili all'utente.

6.1.6 Matrice di tracciabilità

Caso	Requisiti	Caso d'uso	Note
TC01	RF-004, RF-006	UC3	Flusso principale
TC02, TC03, TC12, TC13	RF-004	UC3	Validazione dati obbligatori
TC04, TC18–TC20	RF-006	UC3	Tipologie
TC05	RF-004	UC3	Associazione al progetto
TC06, TC14	RF-005	UC3, UC4	Priorità
TC07, TC21	RF-007	UC10	Tag
TC08, TC15–TC17	RF-005	UC3, UC5	Immagine
TC09	RF-004, RF-005, RF-007	UC3–UC5	Combinazione
TC10, TC11	RF-004, RNF-004	UC3	Errori e atomicità
TC22, TC23, TC24	Vincoli di sicurezza, RNF-007	UC1	Controllo accessi

Tabella 10: Tracciabilità casi di prova → requisiti → casi d'uso

6.2 Test automatici di unità

I test di unità sono due classi JUnit 5, `TestIssueJOOQ` e `TestIssueService`, e seguono la convenzione di naming `metodo_scenario_esito`. I test sono raggruppati per metodo sotto test tramite classi `@Nested` (`NewIssue` e `HandleIssue` in `TestIssueJOOQ`).

Sono test di tipo **white-box**: sono scritti conoscendo l'implementazione della classe sotto test e non si limitano a controllare il valore restituito. Verificano anche aspetti interni, come il numero e l'ordine delle istruzioni SQL eseguite, la presenza di frammenti di SQL (`update`, `assignee_id`, `is null`), i valori passati come binding e le chiamate al DAO. Si distinguono così dal test plan di sistema, che è black-box e si basa solo sugli esiti visibili all'utente.

6.2.1 Metodo 1 e 2: `newIssue(DTO, String)` e `handleIssue(int, String)`

```
1 class TestIssueJOOQ {
2     @Getter
3     static class Executed {
4         private final String sql;
5         private final Object[] bindings;
6
7         Executed(String sql, Object[] bindings) {
8             this.sql = sql;
9             this.bindings = bindings;
10        }
11    }
12
13    private final List<Executed> executed = new ArrayList<>();
14    private IssueJOOQ repository;
15    private int rowsAffected = 1;
16
17    @BeforeEach
18    void setUp() {
19        DSLContext helper = DSL.using(SQLDialect.POSTGRES);
20
21        MockDataProvider provider = ctx -> {
22            executed.add(new Executed(ctx.sql(), ctx.bindings()));
23            String sql = ctx.sql().toLowerCase();
24
25            if (sql.startsWith("select")) {
26                Result<Record1<Integer>> r = helper.newResult(USER_
27                    .USER_ID);
28                r.add(helper.newRecord(USER_.USER_ID).values(42));
29                return new MockResult[] { new MockResult(1, r) };
30            }
31            if (sql.startsWith("insert")) {
32                Result<Record1<Integer>> r = helper.newResult(ISSUE
33                    .ISSUE_ID);
34                r.add(helper.newRecord(ISSUE.ISSUE_ID).values(7));
35                return new MockResult[] { new MockResult(1, r) };
36            }
37        }
38    }
39}
```

```

35         return new MockResult[] { new MockResult(rowsAffected)
36             };
37
38         DSLContext ctx = DSL.using(new MockConnection(provider),
39             SQLDialect.POSTGRES);
40         repository = new IssueJOOQ(ctx);
41     }
42
43     private NewIssueDTO baseDto() {
44         NewIssueDTO dto = new NewIssueDTO();
45         dto.setTitle("Bug");
46         dto.setDescription("desc");
47         dto.setType(IssueType.BUG);
48         dto.setProjectId(1);
49         return dto;
50     }
51
52     @Nested
53     class NewIssue {
54         @Test
55         void newIssue_noOptionalFields_selectNInsert() {
56             repository.newIssue(baseDto(), "a@b.it");
57
58             assertEquals(2, executed.size());
59             assertTrue(executed.get(0).getSql().toLowerCase().
60                 startsWith("select"));
61             assertTrue(executed.get(1).getSql().toLowerCase().
62                 startsWith("insert"));
63         }
64
65         @Test
66         void newIssue_withPriority_updatePriority() {
67             NewIssueDTO dto = baseDto();
68             dto.setPriority(Priority.HIGH);
69
70             repository.newIssue(dto, "a@b.it");
71
72             assertEquals(3, executed.size());
73             Executed update = executed.get(2);
74             assertTrue(update.getSql().toLowerCase().contains("
75                 update"));
76             assertTrue(update.getSql().toLowerCase().contains("
77                 priority"));
78             assertEquals(7, update.getBindings()[update.getBindings
79                 ().length - 1]);
80         }
81
82         @Test
83         void newIssue_withImage_updateImage() {
84             NewIssueDTO dto = baseDto();

```

```

79         dto.setImage("imageBase64String");
80
81         repository.newIssue(dto, "a@b.it");
82
83         assertEquals(3, executed.size());
84         assertTrue(executed.get(2).getSql().toLowerCase().
            contains("image"));
85     }
86
87     @Test
88     void newIssue_withTags_updateTags() {
89         NewIssueDTO dto = baseDto();
90         dto.setTags(List.of("ui", "urgent"));
91
92         repository.newIssue(dto, "a@b.it");
93
94         Executed update = executed.get(2);
95         assertTrue(update.getSql().toLowerCase().contains("tags
            "));
96         assertEquals("ui,urgent", update.getBindings()[0]);
97     }
98
99     @Test
100    void newIssue_allFields_allUpdates() {
101        NewIssueDTO dto = baseDto();
102        dto.setPriority(Priority.LOW);
103        dto.setImage("imageBase64String");
104        dto.setTags(List.of("x"));
105
106        repository.newIssue(dto, "a@b.it");
107
108        assertEquals(5, executed.size());
109    }
110 }
111
112 @Nested
113 class HandleIssue {
114
115     @Test
116     void handleIssue_noRowsAffected_returnFalse() {
117         rowsAffected = 0;
118
119         boolean result = repository.handleIssue(10, "a@b.it");
120
121         assertFalse(result);
122         assertEquals(1, executed.size());
123     }
124
125     @Test
126     void handleIssue_rowsAffected_returnTrue() {
127         rowsAffected = 1;

```

```

128         boolean result = repository.handleIssue(10, "a@b.it");
129
130         assertTrue(result);
131         assertEquals(1, executed.size());
132     }
133
134     @Test
135     void handleIssue_correctSQL() {
136         rowsAffected = 1;
137
138         repository.handleIssue(10, "a@b.it");
139
140         String sql = executed.get(0).getSql().toLowerCase();
141         assertTrue(sql.startsWith("update"));
142         assertTrue(sql.contains("select"));
143         assertTrue(sql.contains("assignee_id"));
144         assertTrue(sql.contains("is null"));
145     }
146
147     @Test
148     void handleIssue_bindValuesWithCorrectParameters() {
149         rowsAffected = 1;
150
151         repository.handleIssue(10, "a@b.it");
152
153         List<Object> b = Arrays.asList(executed.get(0).
154             getBindings());
155         assertTrue(b.contains("a@b.it"));
156         assertTrue(b.contains(10));
157     }
158 }
159 }

```

Listing 1: Unit test con JUnit e Mockito

6.2.2 Metodo 3: getSuggestion(String, int)

```
1 @ExtendWith(MockitoExtension.class)
2 class TestIssueService {
3
4     @Mock
5     private IssueDAO dao;
6
7     private IssueService service;
8
9     @BeforeEach
10    void setUp() {
11        service = new IssueService(dao);
12    }
13
14    private UserDTO user(String email) {
15        UserDTO u = new UserDTO();
16        u.setEmail(email);
17        return u;
18    }
19
20    @Test
21    void getSuggestion_emptyList_returnFalse() {
22        when(dao.getSuggestion(5)).thenReturn(List.of());
23
24        assertFalse(service.getSuggestion("a@b.it", 5));
25    }
26
27    @Test
28    void getSuggestion_emailToSuggest_returnTrue() {
29        when(dao.getSuggestion(5)).thenReturn(List.of(user("x@y.it"),
30            user("a@b.it")));
31
32        assertTrue(service.getSuggestion("a@b.it", 5));
33    }
34
35    @Test
36    void getSuggestion_emailNotToSuggest_returnFalse() {
37        when(dao.getSuggestion(5)).thenReturn(List.of(user("x@y.it"),
38            user("z@y.it")));
39
40        assertFalse(service.getSuggestion("a@b.it", 5));
41    }
42
43    @Test
44    void getSuggestion_emailFirstOccurrence_returnTrue() {
45        when(dao.getSuggestion(3)).thenReturn(List.of(user("a@b.it"),
46            user("x@y.it"), user("z@y.it")));
47
48        assertTrue(service.getSuggestion("a@b.it", 3));
49    }
50 }
```

```

47
48     @Test
49     void getSuggestion_emailLastOccurrence_returnTrue() {
50         when(dao.getSuggestion(3)).thenReturn(List.of(user("x@y.it"
51             ), user("z@y.it"), user("a@b.it")));
52
53         assertTrue(service.getSuggestion("a@b.it", 3));
54     }
55
56     @Test
57     void getSuggestion_topNWorks() {
58         when(dao.getSuggestion(7)).thenReturn(List.of());
59
60         service.getSuggestion("a@b.it", 7);
61
62         verify(dao).getSuggestion(7);
63     }

```

Listing 2: Unit test con JUnit e jOOQ Mock Tools

6.3 Strategie di progettazione dei test

6.3.1 Approccio

White-box. `TestIssueService` è scritto conoscendo la struttura interna di `IssueService`: sa che il servizio riceve un `IssueDAO` nel costruttore e che `getSuggestion(email, n)` delega al DAO la chiamata `getSuggestion(n)`. Il DAO viene sostituito da un mock configurato con `when(...)` sul valore specifico di n , e il test `topNWorks` verifica direttamente l'interazione con il collaboratore tramite `verify`. I casi (lista vuota, email assente, email in posizione intermedia, prima e ultima) sono inoltre scelti in funzione di come la lista restituita dal DAO viene percorsa, non solo di ciò che è specificato dai requisiti.

Tecnica. La classe è eseguita con `MockitoExtension`; `IssueDAO` è un mock (`@Mock`) iniettato nel costruttore di `IssueService`, quindi viene testata la sola logica del servizio, isolata dal livello di persistenza. Il metodo di supporto `user(email)` crea un `UserDTO` con la sola email. Dai test si evince che `getSuggestion(email, n)` interroga il DAO con n e restituisce `true` se l'email compare tra gli utenti restituiti, `false` altrimenti.

Criterio di copertura. I casi sono scelti per partizione dell'input, in base alla presenza dell'email tra gli utenti restituiti dal DAO: lista vuota, email assente, email presente. Per il caso "presente" si considerano la posizione intermedia, la prima e l'ultima. Il test `topNWorks` è un test di interazione (white-box): controlla che il parametro n sia inoltrato al DAO.

6.3.2 Casi limite e valori di confine

White-box. `TestIssueJOOQ` è scritto conoscendo come `IssueJOOQ` è implementata internamente. Il `MockDataProvider` risponde in base al tipo di istruzione (`SELECT` restitui-

sce un utente, INSERT una issue), e questo presuppone che `newIssue` esegua prima una `SELECT` e poi una `INSERT`. I test asseriscono il numero e l'ordine delle istruzioni SQL (2, 3 o 5 a seconda dei campi opzionali), la presenza di frammenti di SQL (`update`, `priority`, `image`, `tags`, `assignee_id`, `is null`) e la posizione dei binding (il primo per i tag, l'ultimo per la priorità). Non si limitano quindi a controllare il risultato osservabile dall'esterno, ma la struttura delle query prodotte.

Tecnica. La classe `IssueJOOQ` viene istanziata con un `DSLContext` collegato a una `MockConnection`: nessuna query raggiunge un database reale. Il setup è composto da:

- la classe di supporto `Executed`, che memorizza il testo SQL e i binding di ogni istruzione eseguita; la lista `executed` permette di contare e ispezionare le istruzioni nell'ordine in cui sono state inviate;
- un `MockDataProvider` che risponde in base al tipo di istruzione: a ogni `SELECT` restituisce un utente con id 42, a ogni `INSERT` una issue con id 7, per qualsiasi altra istruzione un numero di righe modificate pari al campo `rowsAffected` (1 di default, modificabile dal singolo test);
- il metodo `baseDto()`, che costruisce un `NewIssueDTO` minimo (titolo "Bug", descrizione "desc", tipo BUG, progetto 1) a cui i singoli test aggiungono i campi opzionali.

6.3.3 Copertura strutturale

Criterio di copertura. Per `newIssue` i test esercitano i tre campi opzionali (priorità, immagine, tag): nessun campo (base), ciascun campo da solo e tutti insieme, cioè 5 delle $2^3 = 8$ combinazioni possibili. Dal numero di istruzioni osservate (2 senza campi opzionali, 3 con un campo, 5 con tre campi) si evince che ogni campo opzionale comporta un'istruzione aggiuntiva eseguita dopo l'INSERT. Per `handleIssue` vengono coperti entrambi gli esiti del valore di ritorno (nessuna riga modificata e almeno una riga modificata), la struttura dell'SQL e i parametri associati.

Fine del Documento

DOCUMENTAZIONE

INGEGNERIA DEL SOFTWARE

Si ringrazia il lettore per l'attenzione

Mario Majorano	N86005035
Luca Sanselmo	N86005147

Anno Accademico 2025/2026



UNIVERSITÀ DEGLI STUDI
DI NAPOLI FEDERICO II